

```

"""
=====
=====
                                TELEGRAM ADVANCED
REFERRAL BOT SYSTEM
    [ v4: Softer fraud rules (fewer false-
positive bans), stale verify-widget
      cleanup on every /start, and a
      resumable "stop bot" admin control. ]
    [ v4.1: Performance indexes + faster
referral metrics for large user bases ]
    [ One person = One account. Shared
IP alone is NOT a ban reason.    ]
=====
=====
"""

import os
import sys
import json
import hmac
import asyncio
import hashlib
import logging
import urllib.parse
from datetime import datetime
from collections import defaultdict
import time

import httpx

```

```
import uvicorn
import aiosqlite
from dotenv import load_dotenv

from aiogram import Bot, Dispatcher,
Router, F
from aiogram.enums import ParseMode,
ChatMemberStatus
from aiogram.filters import CommandStart
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State,
StatesGroup
from aiogram.fsm.storage.memory import
MemoryStorage
from aiogram.types import (
    Message, CallbackQuery,
    InlineKeyboardMarkup, InlineKeyboardButton,
    WebAppInfo
)

from fastapi import FastAPI, Request,
HTTPException
from fastapi.responses import HTMLResponse,
JSONResponse
from fastapi.middleware.cors import
CORSMiddleware
from contextlib import asynccontextmanager
from pydantic import BaseModel
from typing import Optional, List

load_dotenv()

logging.basicConfig(
    level=logging.INFO,
```

```

        format="%(%asctime)s [%(levelname)s] %(
(name)s: %(message)s",
        handlers=
[logging.StreamHandler(sys.stdout)]
)
logger =
logging.getLogger("ReferralBotSystem")
logger.info("=====
=====")
logger.info("  BOT CODE VERSION: v4.2-
debug-2026-06-30")
logger.info("  If you don't see this line
after deploy/restart,")
logger.info("  the OLD code is still
running on Railway.")
logger.info("=====
=====")

BOT_TOKEN          =
os.getenv("BOT_TOKEN", "")
ADMIN_IDS          = [int(x) for x in
os.getenv("ADMIN_IDS", "0").split(",") if
x.strip()]
PAYMENT_LOG_CHANNEL =
os.getenv("PAYMENT_LOG_CHANNEL",
"").strip()
# WEBAPP_URL = this backend's OWN public
URL (Railway). Used for API calls
# and to build the /verify page link. Kept
for backward compatibility.
WEBAPP_URL         =
os.getenv("WEBAPP_URL",
"http://localhost:8000").rstrip("/")
# FRONTEND_URL = the Mini App static site's

```

```

public URL (Vercel). This is
# what Telegram "Open Mini App" buttons
must point to. Falls back to
# WEBAPP_URL if not set, so old single-
domain deployments still work.
FRONTEND_URL =
os.getenv("FRONTEND_URL", "").rstrip("/")
or WEBAPP_URL
PROXYCHECK_API_KEY =
os.getenv("PROXYCHECK_API_KEY", "")
ALLOWED_ORIGIN =
os.getenv("ALLOWED_ORIGIN", "").strip()
DB_PATH = "referral_bot.db"
TASK_JOIN_WAIT_SECONDS =
int(os.getenv("TASK_JOIN_WAIT_SECONDS",
"5"))

TELEBIRR_PROOF_IMAGE =
"AgACAgQAaxkBAA06akLJQYxDtMsMCF_TJ1mfprGQg9
oAAqg0axv6JBFSsp0Sw79o0x0BAAMCAAN4AAM4BA"

if not WEBAPP_URL.startswith(("http://",
"https://")):
    WEBAPP_URL = f"https://{WEBAPP_URL}"

BOT_RULES_CAPTION = (
    "📜 <b>System Terms of Service & Anti-
Fraud Policy</b>\n\n"
    "1. <b>Strict Integrity:</b> Self-
referrals or multi-accounting schemes are
prohibited.\n"
    "2. <b>Security Protocols:</b> Use of
VPNs, proxy networks, or emulators is
banned.\n"

```

```

    "3. <b>Reward Settlement:</b> Rewards
are credited after Mini App identity
clear.\n"
    "⚠️ <i>Note: Violations will result in
a permanent ban.</i>"
)

#
_____
_____
_____

# RATE LIMITER
#
_____
_____
_____

class RateLimiter:
    def __init__(self, max_calls: int = 5,
window_seconds: int = 60):
        self.max_calls      = max_calls
        self.window         = window_seconds
        self._calls: dict =
defaultdict(list)
        self._lock          = asyncio.Lock()

        async def is_allowed(self, key: str) ->
bool:
            async with self._lock:
                now      = time.monotonic()
                bucket = self._calls[key]
                bucket[:] = [t for t in bucket
if now - t < self.window]
                if len(bucket) >=
self.max_calls:

```

```

        return False
        bucket.append(now)
        return True

verify_limiter = RateLimiter(max_calls=6,
                              window_seconds=60)

#
_____
_____

# IP COOLDOWN – እንድ IP verify ካደረገ በኋላ 3
ደቂቃ ሌላ NEW verify እንዳይሞክር
# (fingerprint ግን ለዘላለም DB ውስጥ ይቆያል – ይህ
  ለ rapid multi-account creation ብቻ ነው)
#
_____
_____

IP_VERIFY_COOLDOWN_SECONDS = 180    # 3 ደቂቃ

class IPCooldownTracker:
    def __init__(self):
        self._last_verify: dict = {}    # ip
        -> timestamp
        self._lock = asyncio.Lock()

        async def is_on_cooldown(self, ip: str)
        -> tuple[bool, int]:
            """Returns (is_blocked,
seconds_remaining)."""
            if not ip or ip in ("127.0.0.1",
":::1", "unknown"):
                return False, 0

```

```

        async with self._lock:
            last =
self._last_verify.get(ip)
            if last is None:
                return False, 0
            elapsed = time.monotonic() -
last
            remaining =
IP_VERIFY_COOLDOWN_SECONDS - elapsed
            if remaining > 0:
                return True, int(remaining)
            return False, 0

    async def mark_verified(self, ip: str):
        if not ip or ip in ("127.0.0.1",
"::1", "unknown"):
            return
        async with self._lock:
            self._last_verify[ip] =
time.monotonic()

ip_cooldown = IPCooldownTracker()

#
_____
_____
_____

# DATABASE SCHEMA
#
_____
_____
_____

SCHEMA = ""
PRAGMA journal_mode=WAL;

```

```

PRAGMA synchronous=NORMAL;

CREATE TABLE IF NOT EXISTS users (
    user_id          INTEGER PRIMARY
KEY,
    username         TEXT,
    full_name        TEXT,
    referred_by      INTEGER,
    balance          REAL    DEFAULT 0,
    is_banned        INTEGER DEFAULT 0,
    last_verify_msg_id INTEGER DEFAULT 0,
    last_seen        TEXT    DEFAULT '',
    joined_at        TEXT    DEFAULT
(datetime('now'))
);

CREATE TABLE IF NOT EXISTS verifications (
    id              INTEGER PRIMARY KEY
AUTOINCREMENT,
    user_id         INTEGER UNIQUE,
    ip_address      TEXT,
    user_agent      TEXT,
    fingerprint     TEXT,
    referrer_ip     TEXT    DEFAULT '',
    tg_platform     TEXT    DEFAULT '',
    tg_version      TEXT    DEFAULT '',
    tg_app_version  TEXT    DEFAULT '',
    canvas_hash     TEXT    DEFAULT '',
    webgl_hash      TEXT    DEFAULT '',
    screen_sig      TEXT    DEFAULT '',
    verified_at     TEXT    DEFAULT
(datetime('now'))
);

```



```

CREATE TABLE IF NOT EXISTS withdrawals (
    id            INTEGER PRIMARY KEY
AUTOINCREMENT,
    user_id       INTEGER,
    amount        REAL,
    full_name     TEXT,
    phone         TEXT,
    status        TEXT    DEFAULT
'pending',
    channel_post_id INTEGER DEFAULT 0,
    reason        TEXT    DEFAULT '',
    created_at    TEXT    DEFAULT
(datetime('now')),
    resolved_at   TEXT
);

```

```

CREATE TABLE IF NOT EXISTS force_channels (
    id            INTEGER PRIMARY KEY
AUTOINCREMENT,
    channel_id    TEXT UNIQUE,
    channel_name  TEXT,
    invite_link   TEXT,
    bot_added     INTEGER DEFAULT 0
);

```

```

CREATE TABLE IF NOT EXISTS settings (
    key    TEXT PRIMARY KEY,
    value  TEXT
);

```

```

CREATE TABLE IF NOT EXISTS fake_join_seen (
    user_id INTEGER PRIMARY KEY,
    seen_at TEXT DEFAULT (datetime('now'))
);

```

```
CREATE TABLE IF NOT EXISTS banned_ips (  
    ip_address TEXT PRIMARY KEY,  
    reason TEXT,  
    banned_at TEXT DEFAULT  
(datetime('now'))  
);
```

```
CREATE TABLE IF NOT EXISTS fraud_log (  
    id INTEGER PRIMARY KEY  
AUTOINCREMENT,  
    user_id INTEGER,  
    reason TEXT,  
    ip_address TEXT,  
    details TEXT DEFAULT '',  
    logged_at TEXT DEFAULT  
(datetime('now'))  
);
```

```
--
```

```
--
```

```
-- MINI APP TASK SYSTEM  
-- task_type: 'force' (bot must be admin in  
channel_id → real membership  
--          check via getChatMember  
before reward is paid)  
--          | 'fake' (no real check – just  
a Join-link + wait-timer, for  
--          channels the bot  
cannot/should not be admin in)  
--
```

```
—
CREATE TABLE IF NOT EXISTS tasks (
  id          INTEGER PRIMARY KEY
AUTOINCREMENT,
  title       TEXT    NOT NULL,
  channel_id  TEXT    DEFAULT '',
  invite_link TEXT    DEFAULT '',
  task_type   TEXT    DEFAULT 'fake',
  reward      REAL    DEFAULT 0,
  position    INTEGER DEFAULT 0,
  is_active   INTEGER DEFAULT 1,
  created_at  TEXT    DEFAULT
(datetime('now'))
);

CREATE TABLE IF NOT EXISTS task_progress (
  user_id     INTEGER,
  task_id     INTEGER,
  status      TEXT DEFAULT 'joined',
  joined_at   TEXT,
  completed_at TEXT,
  PRIMARY KEY (user_id, task_id)
);

CREATE INDEX IF NOT EXISTS
idx_task_progress_user ON
task_progress(user_id);
CREATE INDEX IF NOT EXISTS
idx_tasks_position    ON tasks(position);
```

```
--
```

```
—
```

```
-- PERFORMANCE INDEXES
-- ብዙ users ሲኖሩ balance/referrals/withdraw
ቁልፎች የሚጠቀሙባቸውን query ዎች
-- (referred_by lookups, fingerprint/ip
correlation, withdrawal status)
-- ፈጣን ለማድረግ። IF NOT EXISTS ስለሆነ deploy
በተደጋጋሙ ቁጥር ምንም ችግር አይፈጥርም።
--
```

```
—
CREATE INDEX IF NOT EXISTS
idx_users_referred_by ON
users(referred_by);
CREATE INDEX IF NOT EXISTS
idx_users_is_banned ON
users(is_banned);
CREATE INDEX IF NOT EXISTS
idx_verif_fingerprint ON
verifications(fingerprint);
CREATE INDEX IF NOT EXISTS
idx_verif_ip ON
verifications(ip_address);
CREATE INDEX IF NOT EXISTS
idx_verif_canvas_webgl ON
verifications(canvas_hash, webgl_hash);
CREATE INDEX IF NOT EXISTS
idx_verif_tg_device ON
verifications(tg_platform, tg_version);
CREATE INDEX IF NOT EXISTS
idx_withdrawals_status ON
withdrawals(status);
CREATE INDEX IF NOT EXISTS
idx_withdrawals_user ON
```

```

withdrawals(user_id);
CREATE INDEX IF NOT EXISTS
idx_fraud_log_logged_at ON
fraud_log(logged_at);
CREATE INDEX IF NOT EXISTS
idx_fraud_log_user      ON
fraud_log(user_id);
CREATE INDEX IF NOT EXISTS
idx_banned_ips_ip       ON
banned_ips(ip_address);
CREATE INDEX IF NOT EXISTS
idx_users_last_seen     ON
users(last_seen);

INSERT OR IGNORE INTO settings (key, value)
VALUES ('reward_per_referral', '10');
INSERT OR IGNORE INTO settings (key, value)
VALUES ('min_withdrawal', '50');

INSERT OR IGNORE INTO fake_join_seen
(user_id)
SELECT user_id FROM verifications;
"""

# NOTE: All ALTER TABLE statements are
wrapped in try/except at apply-time
# (see DataEngine.init_database) so they're
safe to keep here permanently –
# running them again on a DB that already
has the columns is a harmless no-op.
MIGRATION_STATEMENTS = [
    "ALTER TABLE verifications ADD COLUMN
referrer_ip    TEXT DEFAULT ''",
    "ALTER TABLE verifications ADD COLUMN

```

```
tg_platform    TEXT DEFAULT '',
    "ALTER TABLE verifications ADD COLUMN
tg_version     TEXT DEFAULT '',
    "ALTER TABLE verifications ADD COLUMN
tg_app_version TEXT DEFAULT '',
    "ALTER TABLE verifications ADD COLUMN
canvas_hash    TEXT DEFAULT '',
    "ALTER TABLE verifications ADD COLUMN
webgl_hash     TEXT DEFAULT '',
    "ALTER TABLE verifications ADD COLUMN
screen_sig     TEXT DEFAULT '',
    "ALTER TABLE users          ADD COLUMN
last_verify_msg_id INTEGER DEFAULT 0",
    "ALTER TABLE users          ADD COLUMN
last_seen TEXT DEFAULT '',
    ""CREATE TABLE IF NOT EXISTS fraud_log
(
    id INTEGER PRIMARY KEY
AUTOINCREMENT,
    user_id INTEGER, reason TEXT,
ip_address TEXT,
    details TEXT DEFAULT '', logged_at
TEXT DEFAULT (datetime('now'))
)""",
]

#
_____
_____
_____

# HTML SANITIZER
#
_____
_____
```

```

def sanitize_html(text: str) -> str:
    if not text:
        return ""
    return (
        text
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("'", "&quot;")
        .replace('"', "&#x27;")
    )

#

```

```

# DATA ENGINE
#

```

```

class DataEngine:
    @staticmethod
    async def init_database():
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.executescript(SCHEMA)
            await db.commit()
            for stmt in
MIGRATION_STATEMENTS:
                try:
                    await db.execute(stmt)
                    await db.commit()

```

```

        except Exception:
            pass
        # Run ANALYZE once at startup
        # so SQLite's query planner picks
        # good index strategies once
        # the table has data.
    try:
        await db.execute("ANALYZE")
        await db.commit()
    except Exception:
        pass

    @staticmethod
    async def get_user(user_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM users WHERE user_id = ?",
(user_id,))
            return await cur.fetchone()

    @staticmethod
    async def create_user(user_id: int,
username: str, full_name: str, referred_by:
int = None):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "INSERT OR IGNORE INTO
users (user_id, username, full_name,
referred_by) VALUES (?, ?, ?, ?)",
                (user_id, username,
full_name, referred_by),

```



```

        )
        await db.commit()

    @staticmethod
    async def add_balance(user_id: int,
amount: float):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE users SET balance =
ROUND(balance + ?, 2) WHERE user_id = ?",
(amount, user_id)
            )
            await db.commit()

    @staticmethod
    async def get_referral_metrics(user_id:
int):
        """
        Direct + tier-2 referral counts.
        Tier-2 now uses an explicit JOIN
instead of a correlated subquery –
        with the idx_users_referred_by
index this lets SQLite do two fast
        index lookups instead of a nested
scan, which matters once the
        users table is large.
        """
        async with
aiosqlite.connect(DB_PATH) as db:
            cur1 = await db.execute("SELECT
COUNT(*) FROM users WHERE referred_by = ?",
(user_id,))
            direct_count = (await

```

```

cur1.fetchone())[0] or 0
        cur2 = await db.execute(
            "SELECT COUNT(*) FROM users
u2 "
            "JOIN users u1 ON
u2.referred_by = u1.user_id "
            "WHERE u1.referred_by = ?",
            (user_id,)
        )
        tier2_count = (await
cur2.fetchone())[0] or 0
        return direct_count,
        tier2_count

    @staticmethod
    async def
get_all_invited_users(user_id: int):
    async with
aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        cur = await db.execute(
            "SELECT user_id, username,
full_name, joined_at FROM users WHERE
referred_by = ?", (user_id,)
        )
        return await cur.fetchall()

    @staticmethod
    async def touch_last_seen(user_id:
int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE users SET last_seen

```

```

        = datetime('now') WHERE user_id = ?",
        (user_id,)
    )
    await db.commit()

    @staticmethod
    async def
get_user_activity_stats(online_window_minut
es: int = 5):
        async with
aiosqlite.connect(DB_PATH) as db:
            cur1 = await db.execute("SELECT
COUNT(*) FROM users")
            total = (await cur1.fetchone())
[0] or 0
            cur2 = await db.execute(
                "SELECT COUNT(*) FROM users
WHERE last_seen != '' "
                f"AND last_seen >=
datetime('now', '-
{int(online_window_minutes)} minutes')")
            )
            online = (await
cur2.fetchone())[0] or 0
            return total, online

    @staticmethod
    async def ban_user(user_id: int,
status: int = 1):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute("UPDATE users
SET is_banned = ? WHERE user_id = ?",
(status, user_id))

```

```

        await db.commit()

    @staticmethod
    async def
full_clear_verification(user_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute("DELETE FROM
verifications WHERE user_id = ?",
(user_id,))
            await db.commit()

    @staticmethod
    async def
inject_fake_verification(user_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "INSERT OR REPLACE INTO
verifications "
                "(user_id, ip_address,
user_agent, fingerprint, referrer_ip,
tg_platform, tg_version, tg_app_version) "
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
                (user_id, "BYPASS_ADMIN",
"BYPASS_ADMIN", f"BYPASS_{user_id}", "",
"", "", ""))
            await db.commit()

    @staticmethod
    async def is_verified(user_id: int) ->
bool:
        async with

```

```

aiosqlite.connect(DB_PATH) as db:
    cur = await db.execute("SELECT
id FROM verifications WHERE user_id = ?",
(user_id,))
    return (await cur.fetchone())
is not None

@staticmethod
async def save_verification(
    user_id: int,
    ip: str,
    ua: str,
    fingerprint: str,
    referrer_ip: str = "",
    tg_platform: str = "",
    tg_version: str = "",
    tg_app_version: str = "",
    canvas_hash: str = "",
    webgl_hash: str = "",
    screen_sig: str = "",
):
    async with
aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "INSERT OR REPLACE INTO
verifications "
            "(user_id, ip_address,
user_agent, fingerprint, referrer_ip, "
            "tg_platform, tg_version,
tg_app_version, "
            "canvas_hash, webgl_hash,
screen_sig) "
            "VALUES
(?,?,?,?,?,?,?,?,?,?)",

```

```

        (user_id, ip, ua,
 fingerprint, referrer_ip,
            tg_platform, tg_version,
tg_app_version,
            canvas_hash, webgl_hash,
screen_sig),
        )
        await db.commit()

    @staticmethod
    async def get_verification(user_id:
int):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM verifications WHERE user_id = ?",
(user_id,))
            return await cur.fetchone()

    @staticmethod
    async def is_ip_banned(ip: str) ->
bool:
        if not ip or ip in ("127.0.0.1",
"::1", "unknown", "BYPASS_ADMIN"):
            return False
        async with
aiosqlite.connect(DB_PATH) as db:
            cur = await db.execute("SELECT
ip_address FROM banned_ips WHERE ip_address
= ?", (ip,))
            return (await cur.fetchone())
    is not None

```

[illegible]

```

        return 0
    async with
aiosqlite.connect(DB_PATH) as db:
        cur = await db.execute(
            "SELECT COUNT(*) FROM
verifications WHERE ip_address = ?", (ip,)
        )
        return (await cur.fetchone())
[0] or 0

    @staticmethod
    async def
count_ip_distinct_fingerprints(ip: str) ->
int:
        """
        How many DIFFERENT device
fingerprints have verified on this IP.
        Used to tell a legitimate shared
network (many distinct phones,
        e.g. CGNAT / office WiFi / mobile
carrier NAT) apart from a real
        bot farm (few/duplicate
fingerprints reused across many accounts).
        """
        if not ip or ip in ("127.0.0.1",
"::1", "unknown", "BYPASS_ADMIN"):
            return 0
        async with
aiosqlite.connect(DB_PATH) as db:
            cur = await db.execute(
                "SELECT COUNT(DISTINCT
fingerprint) FROM verifications "
                "WHERE ip_address = ? AND
fingerprint != ' ' "

```



```

        "AND fingerprint NOT LIKE
'BYPASS_%'",
        (ip,)
    )
    return (await cur.fetchone())
[0] or 0

    @staticmethod
    async def set_last_verify_msg(user_id:
int, msg_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE users SET
last_verify_msg_id = ? WHERE user_id = ?",
(msg_id, user_id)
            )
            await db.commit()

    @staticmethod
    async def create_withdrawal_atomic(
        user_id: int, amount: float,
full_name: str, phone: str
    ) -> tuple[int, bool]:
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute("BEGIN
EXCLUSIVE")
            try:
                cur = await db.execute(
                    "SELECT balance FROM
users WHERE user_id = ?", (user_id,)
                )
                row = await cur.fetchone()

```

```

        if not row or
round(float(row[0]), 2) < round(amount, 2):
            await
db.execute("ROLLBACK")
            return 0, False

        cur2 = await db.execute(
            "INSERT INTO
withdrawals (user_id, amount, full_name,
phone) VALUES (?, ?, ?, ?)",
            (user_id, amount,
full_name, phone),
        )
        tid = cur2.lastrowid
        await db.execute(
            "UPDATE users SET
balance = ROUND(balance - ?, 2) WHERE
user_id = ?",
            (amount, user_id),
        )
        await db.execute("COMMIT")
        return tid, True
    except Exception as e:
        await
db.execute("ROLLBACK")
        raise e

    @staticmethod
    async def get_withdrawal(wid: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM withdrawals WHERE id = ?", (wid,))

```

```

        return await cur.fetchone()

    @staticmethod
    async def update_withdrawal_status(wid:
int, status: str, post_id: int = 0, reason:
str = ""):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE withdrawals SET
status=?, channel_post_id=?, reason=?,
resolved_at=datetime('now') WHERE id=?",
                (status, post_id, reason,
wid),
            )
            await db.commit()

    @staticmethod
    async def get_pending_withdrawals():
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM withdrawals WHERE status='pending'
ORDER BY created_at")
            return await cur.fetchall()

    @staticmethod
    async def add_force_channel(channel_id:
str, channel_name: str, invite_link: str,
bot_added: int = 0):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(

```

```

        "INSERT OR REPLACE INTO
force_channels (channel_id, channel_name,
invite_link, bot_added) VALUES (?, ?, ?, ?)",
        (channel_id, channel_name,
invite_link, bot_added),
        )
        await db.commit()

    @staticmethod
    async def get_force_channels():
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM force_channels")
            return await cur.fetchall()

    @staticmethod
    async def has_seen_fake_join(user_id:
int) -> bool:
        async with
aiosqlite.connect(DB_PATH) as db:
            cur = await db.execute("SELECT
user_id FROM fake_join_seen WHERE user_id =
?", (user_id,))
            return (await cur.fetchone())
is not None

    @staticmethod
    async def mark_fake_join_seen(user_id:
int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute("INSERT OR

```

```

IGNORE INTO fake_join_seen (user_id) VALUES
(?)", (user_id,))
        await db.commit()

        _settings_cache: dict = {}
        _settings_lock = asyncio.Lock()

        @staticmethod
        async def get_setting(key: str,
default=None):
            async with
DataEngine._settings_lock:
                if key in
DataEngine._settings_cache:
                    return
DataEngine._settings_cache[key]
                async with
aiosqlite.connect(DB_PATH) as db:
                    db.row_factory =
aiosqlite.Row
                    cur = await
db.execute("SELECT value FROM settings
WHERE key = ?", (key,))
                    row = await cur.fetchone()
                    val = row["value"] if row
else default

DataEngine._settings_cache[key] = val
                return val

        @staticmethod
        async def set_setting(key: str, value:
str):
            async with

```

```

DataEngine._settings_lock:
    DataEngine._settings_cache[key]
= value
    async with
aiosqlite.connect(DB_PATH) as db:
        await db.execute("INSERT OR
REPLACE INTO settings (key, value) VALUES
(?,?)", (key, value))
        await db.commit()

    @staticmethod
    async def remove_force_channel(row_id:
int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute("DELETE FROM
force_channels WHERE id = ?", (row_id,))
            await db.commit()

    @staticmethod
    async def get_user_withdrawals(user_id:
int, limit: int = 30):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute(
                "SELECT * FROM withdrawals
WHERE user_id=? ORDER BY created_at DESC
LIMIT ?",
                (user_id, limit),
            )
            return await cur.fetchall()

# — TASK SYSTEM

```

```

    @staticmethod
    async def create_task(title: str,
channel_id: str, invite_link: str,
task_type: str, reward: float) -> int:
        async with
aiosqlite.connect(DB_PATH) as db:
            cur = await db.execute("SELECT
COALESCE(MAX(position), -1) + 1 FROM
tasks")
            pos = (await cur.fetchone())[0]
            cur2 = await db.execute(
                "INSERT INTO tasks (title,
channel_id, invite_link, task_type, reward,
position) "
                "VALUES (?, ?, ?, ?, ?, ?)",
                (title, channel_id,
invite_link, task_type, reward, pos),
            )
            await db.commit()
            return cur2.lastrowid

    @staticmethod
    async def get_tasks(active_only: bool =
True):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            q = "SELECT * FROM tasks"
            if active_only:
                q += " WHERE is_active = 1"
            q += " ORDER BY position ASC,
id ASC"

```

```

        cur = await db.execute(q)
        return await cur.fetchall()

    @staticmethod
    async def get_task(task_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM tasks WHERE id = ?", (task_id,))
            return await cur.fetchone()

    @staticmethod
    async def update_task(task_id: int,
**fields):
        if not fields:
            return
        allowed = {"title", "channel_id",
"invite_link", "task_type", "reward",
"is_active", "position"}
        fields = {k: v for k, v in
fields.items() if k in allowed}
        if not fields:
            return
        cols = ", ".join(f"{k} = ?" for k
in fields)
        vals = list(fields.values()) +
[task_id]
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(f"UPDATE tasks
SET {cols} WHERE id = ?", vals)
            await db.commit()

```



```

        @staticmethod
        async def delete_task(task_id: int):
            async with
aiosqlite.connect(DB_PATH) as db:
                await db.execute("DELETE FROM
tasks WHERE id = ?", (task_id,))
                await db.execute("DELETE FROM
task_progress WHERE task_id = ?",
(task_id,))
                await db.commit()

        @staticmethod
        async def reorder_tasks(order: list):
            async with
aiosqlite.connect(DB_PATH) as db:
                for idx, tid in
enumerate(order):
                    await db.execute("UPDATE
tasks SET position = ? WHERE id = ?", (idx,
int(tid)))
                await db.commit()

        @staticmethod
        async def get_task_progress(user_id:
int, task_id: int):
            async with
aiosqlite.connect(DB_PATH) as db:
                db.row_factory = aiosqlite.Row
                cur = await db.execute(
                    "SELECT * FROM
task_progress WHERE user_id = ? AND task_id
= ?", (user_id, task_id)
                )
                return await cur.fetchone()

```

```

        @staticmethod
        async def
get_all_task_progress_for_user(user_id:
int) -> dict:
        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute("SELECT
* FROM task_progress WHERE user_id = ?",
(user_id,))
            rows = await cur.fetchall()
            return {r["task_id"]: r for r
in rows}

```

```

        @staticmethod
        async def mark_task_joined(user_id:
int, task_id: int):
            # INSERT OR IGNORE: the wait-timer
only starts once. Re-tapping
            # "Join" must NOT reset the
countdown (that would let a user farm
            # infinite retries to dodge the
wait).

            async with
aiosqlite.connect(DB_PATH) as db:
                await db.execute(
                    "INSERT OR IGNORE INTO
task_progress (user_id, task_id, status,
joined_at) "
                    "VALUES (?, ?, 'joined',
datetime('now'))",
                    (user_id, task_id),
                )

```

```

        await db.commit()

    @staticmethod
    async def mark_task_completed(user_id:
int, task_id: int):
        async with
aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE task_progress SET
status='completed',
completed_at=datetime('now') "
                "WHERE user_id = ? AND
task_id = ?",
                (user_id, task_id),
            )
            await db.commit()

#
_____
_____

# FRAUD DETECTION ENGINE (v5 -
correlation-based)
#
# ዓላማ: አንድ ሰው = አንድ አካውንት - ግን ንፁ ተጠቃሚን
ላለማጥፋት "ብቻውን ምንም signal ban
# አያደርግም" የሚል መርህ ይከተላል። ይልቁንስ የ NEW ምዝገባ
ምልክቶች ካለፈ ምዝገባ ጋር
# **በሰንት ገለልተኛ ምድቦች ላይ እንደሚገጣጠሙ**
(correlation) እየቆጠረ ነው ውሳኔ
# የሚሰጠው። እያንዳንዱ ምድብ ብቻውን በአጋጣሚ ሊገጣጠም
ይችላል (ተመሳሳይ ስልክ ሞዴል፣
# ተመሳሳይ WiFi፣ cache clear...) - ግን ብዙ
ምድቦች ላይ በተመሳሳይ ጊዜ ከአንድ ሰው ጋር

```

```

# ቢገጣጠሙ ብቻ እውነተኛ clone/multi-account ነው
# ብሎ በእርግጠኝነት መደምደም ይቻላል።
#
# MATCH CATEGORIES (እያንዳንዱ ነጥብ 1 ነው)፦
#   • fingerprint ሙሉ ለሙሉ ይገጣጠማል
#   • canvas_hash + webgl_hash ሁለቱም
#     ይገጣጠማሉ (የ"ሃርድዌር" ምድብ)
#   • ip_address ይገጣጠማል
#   • tg_platform + tg_version ሁለቱም
#     ይገጣጠማሉ
#
# DECISION (ከ ANY ነባር user ጋር ሲነጻጸር)፦
#   • Self-referral (referrer_id ==
#     new_user_id) → BAN (ሁልጊዜ ግልጽ ነው)
#   • Score ≥ 3/4 ምድቦች ከአንድ ተመሳሳይ user ጋር
#     ይገጣጠማሉ → BAN
#   • Score 1-
#     2
#     → LOG ብቻ (admin review)
#   • referrer ጋር ብቻ፡ fingerprint + (IP
#     ወይም hardware)
#     ሁለቱም
#     ይገጣጠማሉ
#       → BAN ("ራሱ ራሱን ጋበዘ")
#   • referrer ጋር fingerprint ብቻ ወይም IP ብቻ
#     ይገጣጠማል → LOG ብቻ
#   • IP farm፡ ብዙ users በአንድ IP ላይ *እና*
#     ጥቂት የተለያዩ
#     fingerprints (ስለዚህ duplicate ስክሪፕት
#     farm ይመስላል) → BAN
#   ብዙ users በአንድ IP ላይ ግን እያንዳንዱ የተለየ
#     fingerprint
#     አለው (ለምሳሌ CGNAT/mobile-data shared
#     IP) → LOG ብቻ፣ ban የለም

```

```

#
# ይሄ ከ v4 የበለጠ ጥንቃቄ ያለው ነው፡- ምንም single
signal (fingerprint ብቻ፣ IP ብቻ፣
# ስልክ ሞዴል ብቻ) ብቻውን ሰውን አያስታግድም፡፡
#

```

```

CORRELATION_BAN_THRESHOLD = 3    # ከ 4ቱ
ምድቦች ቢያንስ 3 ሲገጣጠሙ ብቻ BAN
MAX_USERS_PER_IP              = 10 # ከዚህ በላይ
→ ጠቅሰ ብቻ (review)
MAX_USERS_PER_IP_BAN          = 40 # ከዚህ በላይ
*እና* ዝቅተኛ fingerprint diversity → BAN
IP_FARM_MIN_FP_RATIO          = 0.5 #
distinct_fp / user_count ከዚህ በታች ከሆነ
"duplicate ስክሪፕት" ይመስላል

def _category_match_score(
    fingerprint: str, fp_ok: bool,
    canvas_hash: str, webgl_hash: str,
    hw_ok: bool,
    client_ip: str, ip_ok: bool,
    tg_platform: str, tg_version: str,
    tg_ok: bool,
    row: dict,
) -> tuple[int, list]:
    """Counts how many independent signal
    categories match a single candidate row."""
    score = 0
    matched = []
    if fp_ok and row.get("fingerprint") and

```

```

row["fingerprint"] == fingerprint:
    score += 1
    matched.append("fingerprint")
    if hw_ok and row.get("canvas_hash") ==
canvas_hash and row.get("webgl_hash") ==
webgl_hash:
        score += 1
        matched.append("hardware")
        if ip_ok and row.get("ip_address") and
row["ip_address"] == client_ip:
            score += 1
            matched.append("ip")
        if tg_ok and row.get("tg_platform") ==
tg_platform and row.get("tg_version") ==
tg_version:
            score += 1
            matched.append("tg_device")
    return score, matched

```

```

async def evaluate_clone_risk(
    new_user_id: int,
    referrer_id: int,
    client_ip: str,
    fingerprint: str,
    tg_platform: str = "",
    tg_version: str = "",
    tg_app_version: str = "",
    canvas_hash: str = "",
    webgl_hash: str = "",
    screen_sig: str = "",
) -> tuple[bool, str]:
    """
    Returns (should_ban: bool, reason:

```

```
str).
```

```
    Bans only when multiple independent
    signals correlate to the SAME other
    account. Any single matching signal
    alone is logged for admin review,
    never auto-banned.
    """
```

```
    # — 1. Self-referral
```

```
        if referrer_id and referrer_id ==
new_user_id:
            return True, "self_invite"
```

```
        ip_ok = bool(client_ip) and client_ip
not in ("127.0.0.1", "::1", "unknown")
        fp_ok = bool(fingerprint) and
fingerprint not in ("undefined", "null",
        "")
```

```
        hw_ok = bool(canvas_hash) and
len(canvas_hash) > 8 and bool(webgl_hash)
and len(webgl_hash) > 8
        tg_ok = bool(tg_platform) and
bool(tg_version)
```

```
    # — 2. Correlate against every
candidate that shares ≥1 signal —————
        if fp_ok or hw_ok or ip_ok or tg_ok:
            conditions = []
            params = []
            if fp_ok:
                conditions.append("fingerprint
= ?")
```

```

        params.append(fingerprint)
        if hw_ok:
            conditions.append("(canvas_hash
= ? AND webgl_hash = ?)")
            params.extend([canvas_hash,
webgl_hash])
        if ip_ok:
            conditions.append("ip_address =
?")
            params.append(client_ip)
        if tg_ok:
            conditions.append("(tg_platform
= ? AND tg_version = ?)")
            params.extend([tg_platform,
tg_version])

        query = (
            "SELECT user_id, fingerprint,
canvas_hash, webgl_hash, ip_address, "
            "tg_platform, tg_version FROM
verifications "
            f"WHERE ({' OR
'.join(conditions)}) AND user_id != ?"
        )
        params.append(new_user_id)

        async with
aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            cur = await db.execute(query,
params)
            candidates = await
cur.fetchall()

```



```

        best_score, best_uid, best_matched
= 0, None, []
        for row in candidates:
            score, matched =
_category_match_score(
                fingerprint, fp_ok,
canvas_hash, webgl_hash, hw_ok,
                client_ip, ip_ok,
tg_platform, tg_version, tg_ok, dict(row)
            )
            if score > best_score:
                best_score, best_uid,
best_matched = score, row["user_id"],
matched

        if best_score >=
CORRELATION_BAN_THRESHOLD:
            logger.warning(
                f"[FRAUD] Correlated clone:
new_uid={new_user_id} matches "
                f"uid={best_uid} on
{best_matched} (score={best_score})"
            )
            return True, "correlated_clone"
        elif best_score >= 1:
            logger.warning(
                f"[FRAUD-WARN] Partial
signal match (not banning): "
                f"new_uid={new_user_id}
matches uid={best_uid} on {best_matched}
(score={best_score})"
            )
            await
DataEngine.log_fraud_attempt(

```

```

        new_user_id,
        "partial_signal_match", client_ip,
        f"matches_uid={best_uid}
categories={best_matched} score=
{best_score}"
    )

# — 3. Referrer device check


---


# Needs fingerprint match PLUS (IP
or hardware) match against the
# referrer specifically – a single
matching signal with the referrer
# alone (e.g. just sharing IP, just
sharing fingerprint) is normal
# for family/friends and is only
logged.
if referrer_id and (fp_ok or ip_ok or
hw_ok):
    async with
aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        cur = await db.execute(
            "SELECT ip_address,
fingerprint, canvas_hash, webgl_hash "
            "FROM verifications WHERE
user_id = ?",
            (referrer_id,)
        )
        inv_row = await cur.fetchone()
        if inv_row:
            inv_ip =
inv_row["ip_address"] or ""

```

```

        inv_fp =
inv_row["fingerprint"] or ""
        is_real_ip = inv_ip not in
(" ", "127.0.0.1", "::1", "unknown",
"BYPASS_ADMIN")

        fp_matches_ref = fp_ok and
inv_fp and inv_fp == fingerprint
        ip_matches_ref = ip_ok and
is_real_ip and inv_ip == client_ip
        hw_matches_ref = (
            hw_ok and
inv_row["canvas_hash"] == canvas_hash
            and
inv_row["webgl_hash"] == webgl_hash
        )

        ref_score =
sum([fp_matches_ref, ip_matches_ref,
hw_matches_ref])

        if fp_matches_ref and
(ip_matches_ref or hw_matches_ref):
            return True,
"same_device_as_referrer"
        elif ref_score >= 1:
            logger.warning(
                f"[FRAUD-WARN]
Single-signal match with referrer (not
banning): "
                f"ref={referrer_id}
new={new_user_id} "
                f"fp=
{fp_matches_ref} ip={ip_matches_ref} hw=

```

```

{hw_matches_ref}"
        )
        await
DataEngine.log_fraud_attempt(
            new_user_id,
"referrer_signal_match", client_ip,
            f"referrer=
{referrer_id} fp={fp_matches_ref} ip=
{ip_matches_ref} hw={hw_matches_ref}"
        )

# — 4. IP farm detection – aware of
shared-NAT / CGNAT networks ————
#   A high user count on one IP is, by
itself, completely normal in
#   Ethiopia where mobile carriers use
CGNAT (many real customers
#   share one public IP). We only
treat it as a farm when the SAME IP
#   also shows low fingerprint
diversity – i.e. the same handful of
#   device fingerprints being reused
across many accounts, which is
#   what a scripted farm looks like.
Distinct real phones on a shared
#   network are never banned for this
alone.
    if ip_ok:
        ip_count = await
DataEngine.count_ip_users(client_ip)
        if ip_count >= MAX_USERS_PER_IP:
            distinct_fp = await
DataEngine.count_ip_distinct_fingerprints(c
lient_ip)

```

```

        fp_ratio = (distinct_fp /
ip_count) if ip_count else 1.0
        logger.warning(
            f"[FRAUD-WARN] High IP
usage: ip={client_ip} count={ip_count} "
            f"distinct_fp={distinct_fp}
ratio={fp_ratio:.2f} new_uid={new_user_id}"
        )
        await
DataEngine.log_fraud_attempt(
            new_user_id,
            "high_ip_usage", client_ip,
            f"users={ip_count}
distinct_fp={distinct_fp} ratio=
{fp_ratio:.2f}"
        )
        if ip_count >=
MAX_USERS_PER_IP_BAN and fp_ratio <
IP_FARM_MIN_FP_RATIO:
            return True, "ip_farm"

    return False, ""

def extract_real_ip(request: Request) ->
str:
    cf_ip = request.headers.get("CF-
Connecting-IP", "").strip()
    if cf_ip:
        return cf_ip
    real_ip = request.headers.get("X-Real-
IP", "").strip()
    if real_ip:
        return real_ip

```

```
        forwarded = request.headers.get("X-Forwarded-For", "")
        if forwarded:
            parts = [p.strip() for p in forwarded.split(",") if p.strip()]
            if parts:
                return parts[0]
        if request.client:
            return request.client.host
        return "unknown"
```

```
#
```

```
# WORKFLOW STATES
```

```
#
```

```
class UserWithdrawalWorkflow(StatesGroup):
```

```
    select_payout_gateway = State()
```

```
    input_cash_volume     = State()
```

```
    provide_mobile_digits = State()
```

```
    provide_account_title = State()
```

```
    payout_final_approval = State()
```

```
class AdminConsoleWorkflow(StatesGroup):
```

```
    modify_referral_bounty = State()
```

```
    modify_minimum_cashout = State()
```

```
    append_mandatory_id    = State()
```

```
    append_mandatory_title = State()
```

```
    append_mandatory_url   = State()
```

```

append_noadmin_link      = State()
append_noadmin_title     = State()
direct_balance_target_id = State()
direct_balance_volume    = State()
broadcast_intel_payload   = State()
broadcast_confirmation    = State()
lookup_individual_id      = State()
ban_individual_id         = State()
banish_individual_id      = State()
pardon_individual_full    = State()
pardon_individual_std     = State()
write_reject_reason       = State()

bot      = Bot(token=BOT_TOKEN,
parse_mode=ParseMode.HTML)
dp        =
Dispatcher(storage=MemoryStorage())
core_router = Router()

#
_____
_____
_____

# DEBUG: catch-all callback logger
#
# Registered as a plain middleware-style
outer handler so EVERY incoming
# callback_query is logged before any
filter-specific handler runs. If a
# button tap produces zero output in
Railway logs, this proves the update
# never reached the bot process at all
(Telegram delivery / polling issue,
# not a code-logic bug) – vs. seeing this

```

```
log line but no further handler
# log, which would point at a filter
mismatch instead.
#
_____
_____

@core_router.callback_query.middleware()
async def debug_log_all_callbacks(handler,
event: CallbackQuery, data):
    logger.info(f"[CALLBACK-IN] uid=
{event.from_user.id} data={event.data!r}")
    try:
        return await handler(event, data)
    except Exception as e:
        logger.exception(f"[CALLBACK-ERROR]
uid={event.from_user.id} data=
{event.data!r}: {e}")
        try:
            await event.answer("⚠ Internal
error, please try again.", show_alert=True)
        except Exception:
            pass

#
_____
_____
_____

# FORCE JOIN LOGIC
#
_____
_____
_____

async def
```



```

inspect_compulsory_memberships(user_id:
int) -> list:
    channels = await
DataEngine.get_force_channels()
    unjoined = []
    for ch in channels:
        if ch["bot_added"] == 1:
            continue
        try:
            m = await
bot.get_chat_member(chat_id=ch["channel_id"
], user_id=user_id)
            if m.status in
(ChatMemberStatus.LEFT,
ChatMemberStatus.KICKED,
ChatMemberStatus.RESTRICTED):
                unjoined.append(dict(ch))
        except Exception as e:
            # IMPORTANT: if the bot itself
isn't an admin in this channel
            # (or the channel ID is wrong),
get_chat_member() always throws
            # – and treating that as "user
hasn't joined" locks EVERY
            # regular user out of the bot
forever, on every button, while
            # admins (who bypass this gate
via BYPASS_ADMIN verification)
            # never notice. So a
misconfigured channel must NOT block
            # users; it's logged instead so
it's visible to the admin.
            logger.warning(
                f"[GATE] Could not check

```

```

membership for channel={ch['channel_id']} "
            f"user={user_id}: {e} -
skipping this channel's check "
            f"(bot may not be admin
there)."
        )
        return unjoined

async def enforce_membership_gate(event,
user_id: int) -> bool:
    unjoined = await
inspect_compulsory_memberships(user_id)
    is_callback = isinstance(event,
CallbackQuery)
    current_data = event.data if
is_callback else ""

    if not is_callback or current_data in
("ui_return_home", ""):
        already_seen_fake = await
DataEngine.has_seen_fake_join(user_id)
        if not already_seen_fake:
            all_channels = await
DataEngine.get_force_channels()
            for ch in all_channels:
                if ch["bot_added"] == 1:
                    if not
any(x['channel_id'] == ch['channel_id'] for
x in unjoined):

unjoined.append(dict(ch))

    if not unjoined:
        return True

```

```

        if any(x.get('bot_added') == 1 for x in
unjoined):
            await
DataEngine.mark_fake_join_seen(user_id)

        buttons = []
        for ch in unjoined:

buttons.append([InlineKeyboardButton(text=f
"✚ Join: {ch['channel_name']}",
url=ch["invite_link"])]])

buttons.append([InlineKeyboardButton(text="
✅ Joined / ተቀላቅያለሁ",
callback_data="ui_revalidate_channels"])]

        txt = (
            "👏 <b>Welcome!</b>\n\n"
            "እባክዎ ከታች ያሉትን ሁሉንም ቻናሎች ይቀላቀሉ፤
ከዚያም <b>'Joined'</b> የሚለውን በተን ይጫኑ።\n\n"
            "Please join all channels and
continue."
        )

        if isinstance(event, Message):
            await event.answer(txt,
reply_markup=InlineKeyboardMarkup(inline_ke
yboard=buttons))
        elif isinstance(event, CallbackQuery):
            await event.message.answer(txt,
reply_markup=InlineKeyboardMarkup(inline_ke
yboard=buttons))
            await event.answer()

```

```

        return False

@core_router.callback_query(F.data ==
"ui_revalidate_channels")
async def
process_channel_revalidation(callback:
CallbackQuery, state: FSMContext):
    uid      = callback.from_user.id
    channels = await
DataEngine.get_force_channels()
    real_unjoined = []
    for ch in channels:
        if ch["bot_added"] == 0:
            try:
                m = await
bot.get_chat_member(chat_id=ch["channel_id"
], user_id=uid)
                if m.status in
(ChatMemberStatus.LEFT,
ChatMemberStatus.KICKED,
ChatMemberStatus.RESTRICTED):

real_unjoined.append(ch)
            except Exception as e:
                logger.warning(
                    f"[GATE] Could not
check membership for channel=
{ch['channel_id']} "
                    f"user={uid}: {e} –
skipping this channel's check "
                    f"(bot may not be admin
there).")
    )

```

```

        if real_unjoined:
            return await callback.answer("❌
Please join all channels and continue.",
show_alert=True)

        try:
            await callback.message.delete()
        except Exception:
            pass

        s = await state.get_data()
        ref = s.get("stashed_referrer_id", 0)
        await state.clear()

        if await DataEngine.is_verified(uid):
            await callback.message.answer("✅
Identity clear!",
reply_markup=generate_dashboard_matrix(uid)
)
        else:
            acc = await
DataEngine.get_user(uid)
            if acc and
acc["last_verify_msg_id"]:
                try:
                    await
bot.delete_message(chat_id=uid,
message_id=acc["last_verify_msg_id"])
                except Exception:
                    pass

            sent = await
callback.message.answer(
f"{BOT_RULES_CAPTION}\n\n🔒

```

```

<b>Attestation Step:</b> Launch Mini App
verification:",

reply_markup=generate_verification_widget(u
id, ref, 0)
    )
    await
sent.edit_reply_markup(reply_markup=generat
e_verification_widget(uid, ref,
sent.message_id))
    await
DataEngine.set_last_verify_msg(uid,
sent.message_id)

#
_____
_____
_____

# CORE BOT HANDLERS
#
_____
_____
_____

@core_router.message(CommandStart())
async def process_start_command(message:
Message, state: FSMContext):
    await state.clear()
    uid = message.from_user.id
    args = message.text.split()
    arg = args[1] if len(args) > 1 else ""
    ref = int(arg) if arg.isdigit() and
int(arg) != uid else 0

    acc = await DataEngine.get_user(uid)

```

```

        if acc and acc["is_banned"]:
            return await message.answer("🚫
<b>Banned:</b> Your profile has been
blacklisted.")

        if not await
enforce_membership_gate(message, uid):
            if ref:
                await
state.update_data(stashed_referrer_id=ref)
            return

        if await DataEngine.is_verified(uid):
            return await message.answer("✅
<b>Welcome back!</b> Access granted.",
reply_markup=generate_dashboard_matrix(uid)
)

# — Clean up any stale, unfinished
verification widget —————
# If the user restarts the bot (or taps
an old shared /start link)
# before completing verification, the
previous "Open Mini App" message
# is deleted first and a brand-new one
is sent in its place. This stops
# someone from passing around / re-
using an old verification link and
# keeps the chat from filling up with
duplicate widgets.
        if acc and acc["last_verify_msg_id"]:
            try:
                await
bot.delete_message(chat_id=uid,

```

```

message_id=acc["last_verify_msg_id"])
    except Exception:
        pass

    sent = await message.answer(
        f"{BOT_RULES_CAPTION}\n\n🔒 <b>Next
Step:</b> Verify identity via Mini App:",

reply_markup=generate_verification_widget(u
id, ref, 0)
    )
    await
sent.edit_reply_markup(reply_markup=generat
e_verification_widget(uid, ref,
sent.message_id))
    await
DataEngine.set_last_verify_msg(uid,
sent.message_id)

@core_router.callback_query(F.data ==
"ui_return_home")
async def process_navigation_home(callback:
CallbackQuery, state: FSMContext):
    await state.clear()
    uid = callback.from_user.id
    if not await
DataEngine.is_verified(uid):
        if not await
enforce_membership_gate(callback, uid):
            return
    await callback.message.edit_text(
        "🏠 <b>Main Dashboard Menu / 🇲🇻
🇲🇻</b>",

```



```

reply_markup=generate_dashboard_matrix(uid)
    )

@core_router.callback_query(F.data ==
"ui_fetch_balance")
async def process_balance_query(callback:
CallbackQuery):
    uid = callback.from_user.id
    # Already-verified users joined the
required channels at verification
    # time. Re-checking membership (one
Telegram API call per channel) on
    # every single button tap is what makes
the bot look "stuck" once many
    # users are tapping buttons
concurrently – this skips that re-check.
    if not await
DataEngine.is_verified(uid):
        if not await
enforce_membership_gate(callback, uid):
            return
        acc = await DataEngine.get_user(uid)
        if acc is None:
            # Self-heal: this account has a
verifications row (so is_verified
            # was True) but somehow never got a
users row – most commonly
            # caused by the admin "Full Unban"
path before this fix. Create
            # the missing row instead of
crashing.
            await DataEngine.create_user(
                uid,
                callback.from_user.username or "",

```

```

callback.from_user.full_name or ""
    )
    acc = await
DataEngine.get_user(uid)
    min_w = await
DataEngine.get_setting("min_withdrawal",
"50")
    await callback.message.edit_text(
        f"💰 <b>Your Available Balance:
</b>\n\n"
        f"• Assets: <code>
{float(acc['balance']):.2f} Birr</code>\n"
        f"• Minimum Withdrawal: <code>
{min_w} Birr</code>",

reply_markup=generate_fallback_navigation()
    )

@core_router.callback_query(F.data ==
"ui_fetch_referrals")
async def process_referral_query(callback:
CallbackQuery):
    uid = callback.from_user.id
    if not await
DataEngine.is_verified(uid):
        if not await
enforce_membership_gate(callback, uid):
            return
        direct, _ = await
DataEngine.get_referral_metrics(uid)
        rate = float(await
DataEngine.get_setting("reward_per_referral
", "10"))
        me = await bot.get_me()

```

```

        link =
f"https://t.me/{me.username}?start={uid}"
        await callback.message.edit_text(
            f"👤 <b>Your Referral Network:
</b>\n\n"
            f"• Total Referrals: <b>{direct}
users</b>\n"
            f"• Earnings per Referral: <b>
{rate:.2f} Birr</b>\n"
            f"• Total Earned: <b>{direct *
rate:.2f} Birr</b>\n\n"
            f"🔗 Your link:\n<code>{link}
</code>",

reply_markup=generate_fallback_navigation()
        )

@core_router.callback_query(F.data ==
"ui_fetch_link")
async def process_link_generation(callback:
CallbackQuery):
    uid = callback.from_user.id
    if not await
DataEngine.is_verified(uid):
        if not await
enforce_membership_gate(callback, uid):
            return
    me = await bot.get_me()
    await callback.message.edit_text(
        f"🔗 <b>Your Invite Link:</b>\n\n"
        f"<code>https://t.me/{me.username}?
start={callback.from_user.id}</code>",

reply_markup=generate_fallback_navigation()

```

```

    )

@core_router.message(F.photo)
async def process_get_file_id(message:
Message):
    if not
evaluate_admin_access(message.from_user.id)
:
        return
        file_id = message.photo[-1].file_id
        await message.answer(
            f"📷 <b>File ID Captured:
</b>\n\n<code>{file_id}</code>\n\n"
            f"⚠️ Copy this value and replace
<code>TELEBIRR_PROOF_IMAGE</code> in the
code."
        )

#
_____
_____
_____

# HELPERS
#
_____
_____
_____

def evaluate_admin_access(user_id: int) ->
bool:
    return user_id in ADMIN_IDS

def
parse_telegram_webapp_handshake(init_data:
str) -> dict | None:

```

```

        try:
            parsed =
dict(urllib.parse.parse_qs(init_data,
strict_parsing=True))
            vh      = parsed.pop("hash", "")
            check_str = "\n".join(f"{k}={v}"
for k, v in sorted(parsed.items()))
            key      = hmac.new(b"WebAppData",
BOT_TOKEN.encode(),
hashlib.sha256).digest()
            sig      = hmac.new(key,
check_str.encode(),
hashlib.sha256).hexdigest()
            if not hmac.compare_digest(sig,
vh):
                return None
            return
json.loads(parsed.get("user", "{}"))
        except Exception:
            return None

async def
execute_network_vpn_lookup(client_ip: str)
-> bool:
    """
    VPN CHECK:
    'operator' field (ISP name) is
intentionally ignored – every legitimate
    Ethiopian user is on an ISP like Ethio
Telecom / Safaricom, so keying off
    that field would ban normal users. Only
an explicit VPN/TOR `type` from
    the lookup service counts as a VPN hit.
    """

```

```

        if not client_ip or client_ip in
("127.0.0.1", "::1", "unknown"):
            return False
        try:
            param = f"&key=
{PROXYCHECK_API_KEY}" if PROXYCHECK_API_KEY
else ""
            url =
f"https://proxycheck.io/v2/{client_ip}?
vpn=1&asn=1{param}"
            async with
httpx.AsyncClient(timeout=5) as c:
                r = await c.get(url)
                payload = r.json()
                d = payload.get(client_ip, {})
                ptype = (d.get("type") or
"" ).upper()
                logger.info(f"proxycheck result
ip={client_ip} type={ptype}")
                return ptype in ("VPN", "TOR")
        except Exception:
            logger.warning(f"proxycheck lookup
failed for ip={client_ip}")
            return False

def generate_verification_widget(user_id:
int, ref: int, msg_id: int = 0):
    # index.html (verify screen) is served
from the FRONTEND (Vercel), NOT
    # from this backend, so this must point
at FRONTEND_URL, not WEBAPP_URL.
    url = f"{FRONTEND_URL}/index.html?uid=
{user_id}&ref={ref}&msg_id={msg_id}"
    return

```

```

InlineKeyboardMarkup(inline_keyboard=[[
    InlineKeyboardButton(text="🔓 Open
Mini App & Verify",
web_app=WebAppInfo(url=url))
]])

def generate_app_webapp_button(user_id:
int, admin: bool = False) ->
InlineKeyboardMarkup:
    url = f"{FRONTEND_URL}/app.html?uid=
{user_id}" + ("&admin=1" if admin else "")
    return
InlineKeyboardMarkup(inline_keyboard=[[
    InlineKeyboardButton(text="📱 Open
Mini App / አፑኝ ክፈት",
web_app=WebAppInfo(url=url))
]])

def generate_dashboard_matrix(user_id: int)
-> InlineKeyboardMarkup:
    rows = [
        [InlineKeyboardButton(
            text="📱 Open Mini App / አፑኝ
ክፈት",
            web_app=WebAppInfo(url=f"
{FRONTEND_URL}/app.html?uid={user_id}")
        )],
        [
            InlineKeyboardButton(text="💰
Balance / ሒሳብ",
callback_data="ui_fetch_balance"),
            InlineKeyboardButton(text="👥
Referrals / ጋባዞች",
callback_data="ui_fetch_referrals"),

```

```

        ],
        [
            InlineKeyboardButton(text="🔗
My Link / ᐱᑭክ",
callback_data="ui_fetch_link"),
            InlineKeyboardButton(text="💸
Withdraw / ብር ማውጪ",
callback_data="ui_initiate_withdrawal"),
        ],
    ]
    if evaluate_admin_access(user_id):

rows.append([InlineKeyboardButton(text="⚙️
Admin Control Center",
callback_data="ui_admin_core")])
    return
    InlineKeyboardMarkup(inline_keyboard=rows)

def generate_admin_dashboard() ->
InlineKeyboardMarkup:
    return
    InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(
            text="🌿 Tasks & Full Admin
(Mini App)",
            web_app=WebAppInfo(url=f"
{FRONTEND_URL}/app.html?admin=1")
        )],
        [
            InlineKeyboardButton(text="💎
Set Referral Reward",
callback_data="adm_cmd_reward"),
            InlineKeyboardButton(text="🏠
Set Min Withdrawal",

```



```

callback_data="adm_cmd_min_wd"),
    ],
    [
        InlineKeyboardButton(text="✍️
Edit User Balance",
callback_data="adm_cmd_edit_bal"),
        InlineKeyboardButton(text="📊
Bot Statistics",
callback_data="adm_cmd_stats"),
    ],
    [
        InlineKeyboardButton(text="🔴
Force Join (Real Admin)",
callback_data="adm_cmd_add_mand"),
        InlineKeyboardButton(text="🟡
Fake Join (No Admin)",
callback_data="adm_cmd_add_noadmin"),
    ],
    [InlineKeyboardButton(text="🗑️
Remove Force Channel",
callback_data="adm_cmd_rm_node")],
    [InlineKeyboardButton(text="📋 List
Force Channels",
callback_data="adm_cmd_list_channels")],
    [
        InlineKeyboardButton(text="🍰
Pending Withdrawals",
callback_data="adm_cmd_pending_tickets"),
        InlineKeyboardButton(text="🗣️
Broadcast Message",
callback_data="adm_cmd_broadcast"),
    ],
    [InlineKeyboardButton(text="🔍
Search User",

```

```

callback_data="adm_cmd_search"]],
        [InlineKeyboardButton(text="⚠️
Fraud Log",
callback_data="adm_cmd_fraud_log")],
        [
            InlineKeyboardButton(text="🚫
Ban User",
callback_data="adm_cmd_ban"),
            InlineKeyboardButton(text="✅
Unban Dashboard",
callback_data="adm_cmd_unban_menu"),
        ],
        [InlineKeyboardButton(text="🛑 STOP
BOT ENGINE",
callback_data="adm_stop_bot_confirm1")],
        [InlineKeyboardButton(text="⬅️ Back
to Main Menu",
callback_data="ui_return_home")],
    ])

def
generate_fallback_navigation(target="ui_ret
urn_home") -> InlineKeyboardMarkup:
    return
InlineKeyboardMarkup(inline_keyboard=[[
    InlineKeyboardButton(text="⬅️ Back
/ ተመለስ", callback_data=target)
]])

#
_____
_____
_____

# WITHDRAWAL

```

```

#
_____
_____

@core_router.callback_query(F.data ==
"ui_initiate_withdrawal")
async def
process_withdrawal_start(callback:
CallbackQuery, state: FSMContext):
    uid = callback.from_user.id
    if not await
DataEngine.is_verified(uid):
        if not await
enforce_membership_gate(callback, uid):
            return
        user = await DataEngine.get_user(uid)
        if user is None:
            # Self-heal – see explanation in
process_balance_query above.
            await DataEngine.create_user(
                uid,
callback.from_user.username or "",
callback.from_user.full_name or ""
            )
            user = await
DataEngine.get_user(uid)
            min_w = float(await
DataEngine.get_setting("min_withdrawal",
"50"))
            current_bal =
round(float(user["balance"]), 2)
            if current_bal < min_w:
                return await callback.answer(
                    f"❌ Minimum payout baseline is

```

```

{min_w:.2f} Birr. Your balance is
{current_bal:.2f} Birr.",
        show_alert=True
    )
    await
state.set_state(UserWithdrawalWorkflow.select_payout_gateway)
    await
state.update_data(cached_balance=current_balance, cached_minimum=min_w)
    await callback.message.edit_text(
        "📱 <b>Select Payout Endpoint:
</b>",

reply_markup=InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="📱
Telebirr / ተላብረ",
callback_data="gateway_telebirr")],
        [InlineKeyboardButton(text="❌
Cancel / ሰርዝ",
callback_data="ui_return_home")],
    ])
    )

@core_router.callback_query(F.data ==
"gateway_telebirr",
UserWithdrawalWorkflow.select_payout_gateway)
async def
process_telebirr_selection(callback:
CallbackQuery, state: FSMContext):
    await
state.set_state(UserWithdrawalWorkflow.inpu

```

```

t_cash_volume)
    await callback.message.edit_text(
        "<b>Specify the amount you wish to
withdraw (Birr):</b>",

reply_markup=generate_fallback_navigation()
    )

@core_router.message(UserWithdrawalWorkflow
.input_cash_volume)
async def process_cashout_volume(message:
Message, state: FSMContext):
    s = await state.get_data()
    try:
        val =
round(float(message.text.strip()), 2)
        if val < s["cached_minimum"] or val
> s["cached_balance"]:
            return await message.answer(
                f"❌ Invalid amount.
Minimum withdrawal is
{s['cached_minimum']:.2f} ETB.\n"
                f"⚠️ <b>Your balance is:
</b> {s['cached_balance']:.2f} ETB."
            )
        except Exception:
            return await message.answer(
                f"❌ Please enter a valid
number.\n"
                f"⚠️ <b>Your balance is:</b>
{s['cached_balance']:.2f} ETB."
            )
    await
state.update_data(validated_volume=val)

```

```

        await
state.set_state(UserWithdrawalWorkflow.prov
ide_mobile_digits)
        await message.answer("📱 <b>Provide
Destination Account Mobile Number:</b>")

@core_router.message(UserWithdrawalWorkflow
.provide_mobile_digits)
async def process_mobile_digits(message:
Message, state: FSMContext):
    phone = message.text.strip()
    if len(phone) < 9:
        return await message.answer("❌
Provide a valid mobile number.")
    await
state.update_data(validated_phone=phone)
    await
state.set_state(UserWithdrawalWorkflow.prov
ide_account_title)
        await message.answer("📝 <b>Enter Full
Name of Account Holder:</b>")

@core_router.message(UserWithdrawalWorkflow
.provide_account_title)
async def process_account_title(message:
Message, state: FSMContext):
    title = message.text.strip()
    if len(title) < 3:
        return await message.answer("❌
Name is too short.")
    await
state.update_data(validated_title=title)
    s = await state.get_data()
    safe_title = sanitize_html(title)

```

```

        await message.answer(
            f"⚠️ <b>Review Settlement  

Details</b>\n\n"
            f"• Platform:  

<code>Telebirr</code>\n"
            f"• Amount: <code>{s['validated_volume']:.2f}</code> ETB</code>\n"
            f"• Holder: <code>{safe_title}</code>\n"
            f"• Number: <code>{sanitize_html(s['validated_phone'])}</code>\n\n"
            f"Authorization requested.",

reply_markup=InlineKeyboardMarkup(inline_keyboard=[
    [
        InlineKeyboardButton(text="✅  
Transact Payout",
callback_data="action_payout_dispatch"),
        InlineKeyboardButton(text="❌  
Abort / ሰርዝ",
callback_data="ui_return_home"),
    ]
)
        await
state.set_state(UserWithdrawalWorkflow.payout_final_approval)

async def dispatch_withdrawal_core(uid:
int, amount: float, full_name: str, phone:
str) -> dict:
    """
    Shared withdrawal-submission logic used
    by BOTH the bot-chat flow and

```

the Mini App REST endpoint (/api/withdraw), so there's exactly one place that creates the ledger row and notifies admins.

```
"""
    tid, ok = await
DataEngine.create_withdrawal_atomic(uid,
amount, full_name, phone)
    if not ok:
        return {"status": "error",
"reason": "insufficient_funds"}

    user = await DataEngine.get_user(uid)
    me = await bot.get_me()
    proof_channel_keyboard =
InlineKeyboardMarkup(inline_keyboard=[[
    InlineKeyboardButton(text="🚀
Invite Now",
url=f"https://t.me/{me.username}?start=
{uid}")
]])

    post_id = 0
    if PAYMENT_LOG_CHANNEL:
        try:
            txt = (

f"_____\\n\\n"
                f"🏠 <b>NEW WITHDRAWAL
REQUEST</b>\\n\\n"

f"_____\\n\\n"
                f"👤 <b>Account Holder
Name:</b> {sanitize_html(full_name)}\\n\\n"
```



```

        f"🆔 <b>User ID:</b> <code>
{uid}</code>\n\n"
        f"💰 <b>Requested Amount:
</b> ETB {amount:.2f}\n\n"
        f"📱 <b>Method:</b>
Telebirr Portal\n\n"
        f"📊 <b>Status:</b> Pending
Verification ⌚\n\n"
        f"🕒 <b>Timestamp:</b>
{datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n\n"

f"_____ "

    )
    receipt = await
bot.send_message(PAYMENT_LOG_CHANNEL, txt,
reply_markup=proof_channel_keyboard)
    post_id = receipt.message_id
    await
DataEngine.update_withdrawal_status(tid,
"pending", post_id)
    except Exception as e:
        logger.error(f"Log Channel Post
Error: {e}")

    direct_ref, tier2_ref = await
DataEngine.get_referral_metrics(uid)
    alias_str =
f"@{sanitize_html(user['username'])}" if
user["username"] else "None"
    admin_txt = (
        f"🔥 <b>Incoming Ticket #{tid}
</b>\n"
        f"_____ \n"

```

```

        f"👤 <b>Holder Name:</b>
{sanitize_html(full_name)}\n"
        f"🆔 <b>User ID:</b> <code>{uid}
</code>\n"
        f"👤 <b>Username:</b>
{alias_str}\n"
        f"📞 <b>Phone:</b> <code>
{sanitize_html(phone)}</code>\n"
        f"💰 <b>Amount:</b> <b>{amount:.2f}
Birr</b>\n"
        f"_____ \n"
        f"📊 <b>NETWORK INTEGRITY REPORT:
</b>\n"
        f"• Direct Referrals: <b>
{direct_ref} ሰዎች</b>\n"
        f"• Tier-2 Network Activity: <b>
{tier2_ref} ሰዎች</b>"
    )
    markup =
    InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="✅
Approve (ይጸቅጥ)",
callback_data=f"adm_payout_ap_{tid}"),
            InlineKeyboardButton(text="❌
Deny (ውድቅ አድርግ)",
callback_data=f"adm_payout_rjmenu_{tid}"),
        ],
        [InlineKeyboardButton(text="👤 View
Referrals",
callback_data=f"adm_view_invites_{uid}")]
    ])
    for aid in ADMIN_IDS:
        try:

```

```

        await bot.send_message(aid,
admin_txt, reply_markup=markup)
        except Exception:
            pass

        return {"status": "submitted",
"ticket_id": tid}

@core_router.callback_query(F.data ==
"action_payout_dispatch",
UserWithdrawalWorkflow.payout_final_approva
l)
async def process_payout_dispatch(callback:
CallbackQuery, state: FSMContext):
    s = await state.get_data()
    uid = callback.from_user.id

    result = await
dispatch_withdrawal_core(
        uid, s["validated_volume"],
s["validated_title"], s["validated_phone"]
    )
    if result["status"] == "error":
        return await callback.answer("✗
Insufficient funds.", show_alert=True)

    await state.clear()
    await callback.message.edit_text(
        "📧 <b>Withdrawal Submitted!</b>
Processing within 2-24 hours.",

reply_markup=generate_dashboard_matrix(uid)
    )

```

```

#
_____
_____
_____

# ADMIN – VIEW REFERRALS
#
_____
_____
_____

@core_router.callback_query(F.data.startswi
th("adm_view_invites_"))
async def
process_admin_view_invites(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
):
        return
        target_uid    =
int(callback.data.split("_")[3])
        invited_nodes = await
DataEngine.get_all_invited_users(target_uid
)
        if not invited_nodes:
            return await callback.answer("🚫
This user has not invited anyone yet.",
show_alert=True)
        lines = []
        for idx, node in
enumerate(invited_nodes, 1):
            uname = f"@{node['username']}" if
node['username'] else "No Username"
            lines.append(

```

```

        f"{idx}.
        {sanitize_html(node['full_name'])}
        ({sanitize_html(uname)}) – ID: <code>
        {node['user_id']}</code>\n"
        f"👤17 Joined:
        {node['joined_at']}"
    )
    chunk_txt = f"👤 <b>Invited Users for
    ID {target_uid}</b>\n\n" +
    "\n\n".join(lines)
    if len(chunk_txt) > 4000:
        chunk_txt = chunk_txt[:4000] +
        "\n\n⚠️...List truncated."
    await bot.send_message(
        chat_id=callback.from_user.id,
        text=chunk_txt,

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )
    await callback.answer()

#
_____
_____
_____

# ADMIN – FRAUD LOG
#
_____
_____
_____

@core_router.callback_query(F.data ==
"adm_cmd_fraud_log")
async def process_fraud_log(callback:

```

```

CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
):
        return
    async with aiosqlite.connect(DB_PATH)
as db:
        db.row_factory = aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM fraud_log ORDER
BY logged_at DESC LIMIT 20"
        )
        rows = await cur.fetchall()
        if not rows:
            return await
callback.message.edit_text(
            "🚫 No fraud attempts logged.",
reply_markup=generate_fallback_navigation("
ui_admin_core")
        )
        lines = []
        for r in rows:
            lines.append(
                f"⚠️ <code>{r['reason']}"
</code>\n"
                f"    uid=<code>{r['user_id']}"
</code> ip=<code>{r['ip_address']}"
</code>\n"
                f"    {r['details']}\n"
                f"    🕒 {r['logged_at']}"
            )
        await callback.message.edit_text(
            f"⚠️ <b>Fraud Log (last 20):"
</b>\n\n" + "\n\n".join(lines),

```

```

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

#
_____
_____
_____

# ADMIN -- APPROVE WITHDRAWAL
#
_____
_____
_____

async def approve_withdrawal_core(tid: int)
-> dict:
    """
        Shared approval logic used by BOTH the
        bot-chat inline button and the
        Mini App REST endpoint
        (/api/admin/withdrawals/approve), so
        there's
        exactly one place that posts the
        Telebirr proof photo and notifies
        the user.
    """
    ticket = await
DataEngine.get_withdrawal(tid)
    if not ticket or ticket["status"] !=
"pending":
        return {"ok": False, "reason":
"already_processed"}
    await
DataEngine.update_withdrawal_status(tid,

```

```

"approved", ticket["channel_post_id"])
    me = await bot.get_me()
    proof_channel_keyboard =
InlineKeyboardMarkup(inline_keyboard=[[
    InlineKeyboardButton(text="🚀
Invite Now",
url=f"https://t.me/{me.username}?start=
{ticket['user_id']}")
    ]])
    if PAYMENT_LOG_CHANNEL and
ticket["channel_post_id"]:
        txt = (

f"_____\\n\\n"
        f"✅ <b>WITHDRAWAL
COMPLETED</b>\\n\\n"

f"_____\\n\\n"
        f"👤 <b>Recipient:</b>
{sanitize_html(ticket['full_name'])}\\n\\n"
        f"💰 <b>Amount:</b> ETB
{ticket['amount']:.2f}\\n\\n"
        f"🚀 <b>Operational Registry:
</b> Success ✅\\n\\n"
        f"_____")
    )
    try:
        await bot.send_photo(

chat_id=PAYMENT_LOG_CHANNEL,
        photo=TELEBIRR_PROOF_IMAGE,
        caption=txt,

reply_to_message_id=ticket["channel_post_id"]

```



```

    ],

    reply_markup=proof_channel_keyboard
    )
    except Exception as e:
        logger.error(f"Proof Photo
Error: {e}")
    try:
        await bot.send_message(

chat_id=PAYMENT_LOG_CHANNEL,
            text=txt,

reply_to_message_id=ticket["channel_post_id
"],

reply_markup=proof_channel_keyboard
    )
    except Exception as e2:
        logger.error(f"Proof Text
Fallback Error: {e2}")
    try:
        await bot.send_message(
            ticket["user_id"],
            f"🎉 Your cashout of
{ticket['amount']:.2f} Birr has been
approved and sent via Telebirr!"
        )
    except Exception:
        pass
    return {"ok": True}

@core_router.callback_query(F.data.startswi
th("adm_payout_ap_"))

```

```

async def process_admin_approval(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
):
        return
    tid    = int(callback.data.split("_")
[3])
    result = await
approve_withdrawal_core(tid)
    if not result["ok"]:
        return await
callback.answer("Already processed.")
    await
callback.message.edit_text(callback.message
.text + "\n\n✅ Ticket Approved.")

#
_____
_____
_____

# ADMIN – REJECT WITHDRAWAL
#
_____
_____
_____

@core_router.callback_query(F.data.startswi
th("adm_payout_rjmenu_"))
async def
process_admin_rejection_menu(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
):

```

```

        return
        tid = int(callback.data.split("_")[3])
        markup =
InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(text="🤖
Multi-bot / Clone Account",
callback_data=f"rj_select_{tid}_multi_bot")
],
            [InlineKeyboardButton(text="❌ Fake
Activity / Emulators",
callback_data=f"rj_select_{tid}_fake_activi
ty")],
            [InlineKeyboardButton(text="👥 No
Organic Invites",
callback_data=f"rj_select_{tid}_no_invites"
)],
            [InlineKeyboardButton(text="✍️
Write Custom Reason",
callback_data=f"rj_select_{tid}_write_custo
m")],
            [InlineKeyboardButton(text="⬅️
Cancel",
callback_data="ui_admin_core")],
        ])
        await callback.message.edit_text("❌
<b>Select Rejection Reason:</b>",
reply_markup=markup)

@core_router.callback_query(F.data.startswi
th("rj_select_"))
async def
process_reason_selection(callback:
CallbackQuery, state: FSMContext):
    if not

```

```

evaluate_admin_access(callback.from_user.id
):
    return
    parts = callback.data.split("_")
    tid = int(parts[2])
    choice = "_".join(parts[3:])
    ticket = await
DataEngine.get_withdrawal(tid)
    if not ticket or ticket["status"] !=
"pending":
        return await
callback.answer("Already processed.")
    if choice == "write_custom":
        await
state.set_state(AdminConsoleWorkflow.write_
reject_reason)
        await
state.update_data(active_reject_tid=tid)
        return await
callback.message.edit_text("👉 <b>Write the
rejection reason to send to user:</b>")
        reason_map = {
            "multi_bot": "Multi-account /
Clone system detected.",
            "fake_activity": "Fake verification
/ Fraudulent activity detected.",
            "no_invites": "Insufficient
organic or active referrals.",
        }
        reason = reason_map.get(choice,
"Violated bot usage policies.")
        await
execute_withdrawal_rejection(callback.messa
ge, tid, ticket, reason)

```

```

@core_router.message(AdminConsoleWorkflow.w
rite_reject_reason)
async def
process_custom_written_reason(message:
Message, state: FSMContext):
    s = await state.get_data()
    await state.clear()
    tid    = s["active_reject_tid"]
    ticket = await
DataEngine.get_withdrawal(tid)
    if not ticket or ticket["status"] !=
"pending":
        return await message.answer("Ticket
already processed.")
    await
execute_withdrawal_rejection(message, tid,
ticket, message.text.strip())

async def
execute_withdrawal_rejection(msg_obj, tid,
ticket, reason):
    await
DataEngine.update_withdrawal_status(tid,
"rejected", ticket["channel_post_id"],
reason)
    # Refund the reserved balance back to
the user – it was deducted
    # atomically at request time in
create_withdrawal_atomic().
    await
DataEngine.add_balance(ticket["user_id"],
float(ticket["amount"]))
    warning_notice = (

```

```

        f"❌ <b>Your Withdrawal Request has
been Rejected!</b>\n\n"
        f"💰 <b>Amount:</b> <code>
{ticket['amount']:.2f} Birr</code>\n"
        f"⚠️ <b>Reason:</b> <code>
{sanitize_html(reason)}</code>\n\n"
        f"_____ \n"
        f"🔊 <b>IMPORTANT NOTICE:</b>\n"
        f"🇬🇧 Please invite real, organic,
and active users.\n\n"
        f"🇲🇪 እባክዎ እውነተኛ ተጠቃሚዎችን ብቻ
ይጋብዙ::\n"
        f"_____"
    )
    try:
        await
        bot.send_message(chat_id=ticket["user_id"],
        text=warning_notice)
    except Exception:
        pass
    reply_text = f"✅ Ticket #{tid}
rejected. User notified."
    if msg_obj is None:
        return
    if isinstance(msg_obj, Message):
        await msg_obj.answer(reply_text,
        reply_markup=generate_admin_dashboard())
    else:
        await msg_obj.edit_text(reply_text,
        reply_markup=generate_admin_dashboard())

#
_____
_____

```

```
# ADMIN – PANEL & SETTINGS
```

```
#
```

```
@core_router.callback_query(F.data ==
"ui_admin_core")
async def process_admin_panel(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await callback.message.edit_text("⚙️
<b>Operational Admin Master Engine</b>",
reply_markup=generate_admin_dashboard())
```

```
@core_router.callback_query(F.data ==
"adm_cmd_add_mand")
async def
process_add_channel_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.append
_mandatory_id)
    await callback.message.edit_text(
        "🔴 <b>Force Join – Real
Check</b>\nEnter Channel ID (e.g.
<code>-1001234567890</code>):",
reply_markup=generate_fallback_navigation("
```

```

ui_admin_core")
    )

@core_router.message(AdminConsoleWorkflow.append_mandatory_id)
async def process_add_channel_id(message: Message, state: FSMContext):
    await
    state.update_data(ch_id=message.text.strip())
    await
    state.set_state(AdminConsoleWorkflow.append_mandatory_title)
    await message.answer("✎ <b>Enter Channel Display Title:</b>")

@core_router.message(AdminConsoleWorkflow.append_mandatory_title)
async def
process_add_channel_title(message: Message, state: FSMContext):
    await
    state.update_data(ch_title=message.text.strip())
    await
    state.set_state(AdminConsoleWorkflow.append_mandatory_url)
    await message.answer("🔗 <b>Enter Channel Invite Link:</b>")

@core_router.message(AdminConsoleWorkflow.append_mandatory_url)
async def
process_add_channel_finalize(message:

```



```

Message, state: FSMContext):
    s = await state.get_data()
    await state.clear()
    await
DataEngine.add_force_channel(s["ch_id"],
s["ch_title"], message.text.strip(),
bot_added=0)
    await message.answer(f"✅ <b>Force
channel added!</b>\n🔴
{sanitize_html(s['ch_title'])}",
reply_markup=generate_admin_dashboard())

@core_router.callback_query(F.data ==
"adm_cmd_add_noadmin")
async def
process_add_noadmin_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.append
_noadmin_link)
    await callback.message.edit_text(
        "🟡 <b>Fake Join (No Admin
Required)</b>\n\n👤 ለንክ አስገባ:",
reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

@core_router.message(AdminConsoleWorkflow.a
ppend_noadmin_link)
async def process_noadmin_link(message:

```

```

Message, state: FSMContext):
    await
    state.update_data(na_link=message.text.strip())
    await
    state.set_state(AdminConsoleWorkflow.append_noadmin_title)
    await message.answer("✎ <b>Enter Display Name:</b>")

@core_router.message(AdminConsoleWorkflow.append_noadmin_title)
async def process_noadmin_title(message: Message, state: FSMContext):
    s = await state.get_data()
    await state.clear()
    fake_key = "fake_" +
    hashlib.md5(s["na_link"].encode()).hexdigest()[:8]
    await DataEngine.add_force_channel(
        channel_id=fake_key,
        channel_name=message.text.strip(),
        invite_link=s["na_link"], bot_added=1
    )
    await message.answer(f"✅ <b>Fake ቻናል ተጨምሯል!</b>\n📌
    {sanitize_html(message.text.strip())}",
        reply_markup=generate_admin_dashboard())

@core_router.callback_query(F.data == "adm_cmd_list_channels")
async def process_list_channels(callback: CallbackQuery):
    if not

```

```

evaluate_admin_access(callback.from_user.id
): return
    channels = await
DataEngine.get_force_channels()
    if not channels:
        return await
callback.message.edit_text("🚫 No
channels.",
reply_markup=generate_fallback_navigation("
ui_admin_core"))
    lines = []
    for ch in channels:
        mode = "🟡 Fake" if ch["bot_added"]
else "🔴 Real"
        lines.append(f"{mode} – <b>
{sanitize_html(ch['channel_name'])}</b>\n🔗
{ch['invite_link']}")
    await callback.message.edit_text(
        f"📋 <b>Force Channels
({len(channels)})</b>\n\n" +
"\n\n".join(lines),

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

@core_router.callback_query(F.data ==
"adm_cmd_rm_node")
async def process_rm_channel_menu(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    channels = await

```

```

DataEngine.get_force_channels()
    if not channels:
        return await
callback.message.edit_text("🚫 No
channels.",
reply_markup=generate_fallback_navigation("
ui_admin_core"))
    buttons = []
    for ch in channels:
        mode = "🟡" if ch["bot_added"] else
"🔴"

buttons.append([InlineKeyboardButton(text=f
"🗑️ {mode}
{sanitize_html(ch['channel_name'])}",
callback_data=f"execute_rm_node_{ch['id']}"
)])

buttons.append([InlineKeyboardButton(text="
🔙 Back", callback_data="ui_admin_core")])
    await callback.message.edit_text("
<b>Select channel to remove:</b>",
reply_markup=InlineKeyboardMarkup(inline_ke
yboard=buttons))

@core_router.callback_query(F.data.startswi
th("execute_rm_node_"))
async def
process_rm_channel_action(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    row_id =

```

```

int(callback.data.replace("execute_rm_node_
", ""))
    async with aiosqlite.connect(DB_PATH)
as db:
    await db.execute("DELETE FROM
force_channels WHERE id = ?", (row_id,))
    await db.commit()
    await callback.message.edit_text("✅
Channel removed.",
reply_markup=generate_admin_dashboard())

#


---




---




---



# ADMIN – STOP / RESUME BOT ENGINE
#
# Stopping the bot ONLY stops Telegram
long-polling (no new /start, button
# taps, or admin replies are processed). It
does NOT touch the SQLite
# database, the Mini App, or any stored
balances/verifications/withdrawals –
# all of that data is safe on disk the
entire time. The "Resume Polling"
# button below restarts the polling task in
the same running process, so an
# admin can pause and resume the bot
without losing anything or having to
# redeploy – as long as the underlying
process/server itself stays alive.
#


---




---




---



```

```

_polling_task: asyncio.Task | None = None
BOT_USERNAME: str = ""

@core_router.callback_query(F.data ==
"adm_stop_bot_confirm1")
async def
stop_bot_first_confirmation(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    markup =
InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="⚠
ኃላፊነቱን እወስዳለሁ - ቀጥል",
callback_data="adm_stop_bot_confirm2")],
        [InlineKeyboardButton(text="❌
አቁም/ተመለስ",
callback_data="ui_admin_core")],
    ])
    await callback.message.edit_text(
        "🚨 <b>FIRST WARNING!</b>\n\n"
        "ጦቱን ማቆም ከፈለጉ እርግጠኛ ነዎት? (ይህ የ
chat polling ብቻ ያቆማል – መረጃ/ሒሳብ አይጠፋም)",
        reply_markup=markup
    )

@core_router.callback_query(F.data ==
"adm_stop_bot_confirm2")
async def
stop_bot_final_confirmation(callback:
CallbackQuery):
    if not

```

```

evaluate_admin_access(callback.from_user.id
): return
    markup =
InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(text="🛑 አሁኑኑ
ቦታ ይቋም!",
callback_data="adm_stop_bot_execute")],
    [InlineKeyboardButton(text="❌
ተመለስ",
callback_data="ui_admin_core")],
    ])
    await callback.message.edit_text("🛑
<b>FINAL CONFIRMATION!</b>",
reply_markup=markup)

@core_router.callback_query(F.data ==
"adm_stop_bot_execute")
async def execute_bot_shutdown(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await callback.message.edit_text(
        "🛑 <b>Bot polling stopped.
</b>\n\n"
        "✅ All data, balances, and Mini
App verification are safe and untouched.\n"
        "▶ Tap below to resume any time
(process must still be running).",

reply_markup=InlineKeyboardMarkup(inline_ke
yboard=[[
    InlineKeyboardButton(text="▶
Resume Polling / ቀጥል",

```

```

        callback_data="adm_resume_bot_execute")
    ])
)
await dp.stop_polling()

@core_router.callback_query(F.data ==
"adm_resume_bot_execute")
async def execute_bot_resume(callback:
CallbackQuery):
    global _polling_task
    if not
evaluate_admin_access(callback.from_user.id
): return
    if _polling_task is None or
_polling_task.done():
        _polling_task =
asyncio.create_task(dp.start_polling(bot,
skip_updates=True))
    await callback.message.edit_text(
        "✅ <b>Bot polling resumed.</b> All
data is intact.",

reply_markup=generate_admin_dashboard()
)

@core_router.callback_query(F.data ==
"adm_cmd_edit_bal")
async def
process_edit_balance_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await

```



```

state.set_state(AdminConsoleWorkflow.direct
_balance_target_id)
    await callback.message.edit_text("<b>Enter Targeted Telegram User ID:</b>",
reply_markup=generate_fallback_navigation("
ui_admin_core"))

@core_router.message(AdminConsoleWorkflow.d
irect_balance_target_id)
async def process_edit_balance_id(message:
Message, state: FSMContext):
    await
state.update_data(target_uid=int(message.te
xt.strip()))
    await
state.set_state(AdminConsoleWorkflow.direct
_balance_volume)
    await message.answer("<b>Enter
Adjustment Volume (e.g. 50 or -50):</b>")

@core_router.message(AdminConsoleWorkflow.d
irect_balance_volume)
async def
process_edit_balance_final(message:
Message, state: FSMContext):
    s = await state.get_data()
    await state.clear()
    await
DataEngine.add_balance(s["target_uid"],
float(message.text.strip()))
    await message.answer("✅ Balance
adjusted.",
reply_markup=generate_admin_dashboard())

```

```

@core_router.callback_query(F.data ==
"adm_cmd_stats")
async def process_stats(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    async with aisqlite.connect(DB_PATH)
as db:
        cur = await db.execute("SELECT
COUNT(*), SUM(balance) FROM users")
        row = await cur.fetchone()
        cur2 = await db.execute("SELECT
COUNT(*) FROM banned_ips")
        ip_count = (await cur2.fetchone())
[0] or 0
        cur3 = await db.execute("SELECT
COUNT(*) FROM fraud_log")
        fraud_count = (await
cur3.fetchone())[0] or 0
        await callback.message.edit_text(
            f" <b>Bot Analytics:</b>\n\n"
            f"• Registered Users: <b>{row[0] or
0}</b>\n"
            f"• Outstanding Liabilities: <b>
{float(row[1] or 0.0):.2f} ETB</b>\n"
            f"• Banned IPs: <b>{ip_count}
</b>\n"
            f"• Fraud Log Entries: <b>
{fraud_count}</b>",

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

```

```

@core_router.callback_query(F.data ==
"adm_cmd_search")
async def process_search_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.lookup
_individual_id)
    await callback.message.edit_text("🔍
<b>Enter Target User Telegram ID:</b>",
reply_markup=generate_fallback_navigation("
ui_admin_core"))

@core_router.message(AdminConsoleWorkflow.l
ookup_individual_id)
async def process_search_execute(message:
Message, state: FSMContext):
    await state.clear()
    try:
        target = int(message.text.strip())
    except ValueError:
        return await message.answer("❌
Invalid ID.",
reply_markup=generate_admin_dashboard())
    user = await
DataEngine.get_user(target)
    if not user:
        return await message.answer("❌
User not found.",
reply_markup=generate_admin_dashboard())
    direct, tier2 = await

```

```

DataEngine.get_referral_metrics(target)
    verif    = await
DataEngine.get_verification(target)
    ip_info  = f"<code>
{verif['ip_address']}</code>" if verif
else "Not verified"
    ref_ip   = f"<code>
{verif['referrer_ip']}</code>" if (verif
and verif['referrer_ip']) else "N/A"
    tg_plat  = (verif['tg_platform']    or
"N/A") if verif else "N/A"
    tg_ver   = (verif['tg_version']     or
"N/A") if verif else "N/A"
    tg_app   = (verif['tg_app_version'] or
"N/A") if verif else "N/A"
    canvas   = (verif['canvas_hash'][:16] +
"..." if verif and verif.get('canvas_hash')
else "N/A")
    webgl    = (verif['webgl_hash'][:16] +
"..." if verif and verif.get('webgl_hash')
else "N/A")

    async with aisqlite.connect(DB_PATH)
as db:
        db.row_factory = aisqlite.Row
        cur = await db.execute(
            "SELECT reason, logged_at FROM
fraud_log WHERE user_id = ? ORDER BY
logged_at DESC LIMIT 3",
            (target,)
        )
        fraud_rows = await cur.fetchall()
        fraud_txt = ""
        if fraud_rows:

```

```

        fraud_txt = "\n🚨 <b>Fraud Flags:
</b>\n" + "\n".join(
            f" • <code>{r['reason']}]
</code> @ {r['logged_at']}" for r in
fraud_rows
        )

    await message.answer(
        f"👤 <b>User Profile:</b>\n\n"
        f"• Name:
{sanitize_html(user['full_name'])}\n"
        f"• Alias:
@{sanitize_html(user['username'] or
'N/A')}\n"
        f"• Balance: <b>
{float(user['balance']):.2f} Birr</b>\n"
        f"• Direct Invites: <b>{direct}
</b>\n"
        f"• Tier-2 Network: <b>{tier2}
</b>\n"
        f"• Banned: <b>{'Yes' if
user['is_banned'] else 'No'}</b>\n"
        f"• IP Address: {ip_info}\n"
        f"• Referrer IP: {ref_ip}\n"
        f"• TG Platform: <b>{tg_plat}
</b>\n"
        f"• TG Version: <b>{tg_ver}</b>\n"
        f"• TG App Version: <b>{tg_app}
</b>\n"
        f"• Canvas Hash: <code>{canvas}
</code>\n"
        f"• WebGL Hash: <code>{webgl}
</code>\n"
        f"• Joined: {user['joined_at']}"
    )

```

```

        f"{fraud_txt}",

reply_markup=generate_admin_dashboard()
    )

@core_router.callback_query(F.data ==
"adm_cmd_ban")
async def process_ban_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.ban_in
dividual_id)
    await callback.message.edit_text(
        "🚫 <b>Ban User</b>\n\nEnter the
Telegram User ID to ban:",

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

@core_router.message(AdminConsoleWorkflow.b
an_individual_id)
async def process_ban_execute(message:
Message, state: FSMContext):
    await state.clear()
    try:
        target = int(message.text.strip())
    except ValueError:
        return await message.answer("❌
Invalid ID.",
reply_markup=generate_admin_dashboard())

```

```

        await DataEngine.ban_user(target, 1)
    try:
        await bot.send_message(target, "🚫
<b>Your account has been banned from this
bot.</b>")
    except Exception:
        pass
    await message.answer(f"✅ User <code>
{target}</code> has been banned.",
reply_markup=generate_admin_dashboard())

@core_router.callback_query(F.data ==
"adm_cmd_reward")
async def process_reward_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.modify
_referral_bounty)
    await callback.message.edit_text("
<b>Enter New Reward Per Referral (Birr):
</b>",
reply_markup=generate_fallback_navigation("
ui_admin_core"))

@core_router.message(AdminConsoleWorkflow.m
odify_referral_bounty)
async def process_reward_execute(message:
Message, state: FSMContext):
    await
DataEngine.set_setting("reward_per_referral
", message.text.strip())

```

```

        await state.clear()
        await message.answer("✅ Bounty
updated.",
reply_markup=generate_admin_dashboard())

@core_router.callback_query(F.data ==
"adm_cmd_min_wd")
async def process_min_wd_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.modify
_minimum_cashout)
    await callback.message.edit_text("
<b>Enter New Minimum Cashout Threshold
(Birr):</b>",
reply_markup=generate_fallback_navigation("
ui_admin_core"))

@core_router.message(AdminConsoleWorkflow.m
odify_minimum_cashout)
async def process_min_wd_execute(message:
Message, state: FSMContext):
    await
DataEngine.set_setting("min_withdrawal",
message.text.strip())
    await state.clear()
    await message.answer("✅ Minimum
withdrawal updated.",
reply_markup=generate_admin_dashboard())

@core_router.callback_query(F.data ==

```



```

"adm_cmd_pending_tickets")
async def
process_pending_inventory(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    pending = await
DataEngine.get_pending_withdrawals()
    if not pending:
        return await
callback.message.edit_text("🔒 No pending
withdrawals.",
reply_markup=generate_fallback_navigation("
ui_admin_core"))
    lines = [f"• <b>#{t['id']}</b> –
{sanitize_html(t['full_name'])} – <code>
{float(t['amount']):.2f} ETB</code>" for t
in pending]
    await callback.message.edit_text(
        f"🔒 <b>Pending Withdrawals
({len(pending)})</b>\n\n" +
        "\n".join(lines),

reply_markup=generate_fallback_navigation("
ui_admin_core")
    )

@core_router.callback_query(F.data ==
"adm_cmd_broadcast")
async def process_broadcast_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id

```

```

): return
    await
state.set_state(AdminConsoleWorkflow.broadcast_intel_payload)
    await callback.message.edit_text("🔔🔔
<b>Enter Broadcast Message:</b>",
reply_markup=generate_fallback_navigation("ui_admin_core"))

@core_router.message(AdminConsoleWorkflow.broadcast_intel_payload)
async def
process_broadcast_preview(message: Message,
state: FSMContext):
    text = message.text
    await
state.update_data(bc_payload=text)
    await
state.set_state(AdminConsoleWorkflow.broadcast_confirmation)
    markup =
InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🚀 Send
Now", callback_data="bc_action_confirm")],
        [InlineKeyboardButton(text="✎
Edit",
callback_data="adm_cmd_broadcast")],
        [InlineKeyboardButton(text="❌
Cancel",
callback_data="ui_admin_core")],
    ])
    await message.answer(f"✎🔔 <b>Preview:
</b>\n\n{text}\n\n⚠️ Send to all users?",
reply_markup=markup)

```

```

@core_router.callback_query(F.data ==
"bc_action_confirm",
AdminConsoleWorkflow.broadcast_confirmation
)
async def
process_broadcast_execute(callback:
CallbackQuery, state: FSMContext):
    s    = await state.get_data()
    text = s["bc_payload"]
    await state.clear()
    progress = await
callback.message.edit_text("⌚ Sending
broadcast...")
    async with aisqlite.connect(DB_PATH)
as db:
        cur    = await db.execute("SELECT
user_id FROM users")
        nodes = await cur.fetchall()
        sent_count = 0
        fail_count = 0
        for (uid,) in nodes:
            try:
                await bot.send_message(uid,
text)
                sent_count += 1
                await asyncio.sleep(0.05)
            except Exception as e:
                err_str = str(e)
                if "RetryAfter" in err_str:
                    try:
                        wait =
int('').join(filter(str.isdigit, err_str))
+ 1

```

```

        except Exception:
            wait = 30
            await asyncio.sleep(wait)
        try:
            await
bot.send_message(uid, text)
            sent_count += 1
        except Exception:
            fail_count += 1
    else:
        fail_count += 1
    try:
        await progress.delete()
    except Exception:
        pass
    await callback.message.answer(
        f"✅ Broadcast complete.\n• Sent: {sent_count}\n• Failed: {fail_count}",
        reply_markup=generate_admin_dashboard()
    )

@core_router.callback_query(F.data ==
"adm_cmd_unban_menu")
async def process_unban_dashboard(callback:
CallbackQuery):
    if not
evaluate_admin_access(callback.from_user.id
): return
    markup =
InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="✅
Standard Unban (Requires MiniApp)",
callback_data="unban_trigger_std")],

```

```

        [InlineKeyboardButton(text="🔥 Full
Unban (Bypass MiniApp)",
callback_data="unban_trigger_full")],
        [InlineKeyboardButton(text="⬅️
Back",
callback_data="ui_admin_core")],
    ])
    await callback.message.edit_text("🔒
<b>Select Unban Method:</b>",
reply_markup=markup)

@core_router.callback_query(F.data ==
"unban_trigger_std")
async def process_std_unban_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.pardon
_individual_std)
    await callback.message.edit_text(
        "👤 <b>[Standard Unban]</b> Enter
Telegram User ID:",

reply_markup=generate_fallback_navigation("
adm_cmd_unban_menu")
    )

@core_router.message(AdminConsoleWorkflow.p
ardon_individual_std)
async def
process_std_unban_execute(message: Message,
state: FSMContext):

```

```

        try:
            target = int(message.text.strip())
            await DataEngine.ban_user(target,
0)
            await
DataEngine.full_clear_verification(target)
            await state.clear()
            await message.answer(
                f"✅ Standard Unban done. User
<code>{target}</code> must reverify via
Mini App.",

reply_markup=generate_admin_dashboard()
            )
        except ValueError:
            await message.answer("❌ Invalid
ID.")

@core_router.callback_query(F.data ==
"unban_trigger_full")
async def
process_full_unban_start(callback:
CallbackQuery, state: FSMContext):
    if not
evaluate_admin_access(callback.from_user.id
): return
    await
state.set_state(AdminConsoleWorkflow.pardon
_individual_full)
    await callback.message.edit_text(
        "🔥 <b>[Full Unban]</b> Enter
Telegram User ID:",

reply_markup=generate_fallback_navigation("

```

```

adm_cmd_unban_menu")
    )

@core_router.message(AdminConsoleWorkflow.pardon_individual_full)
async def process_full_unban_execute(message: Message, state: FSMContext):
    try:
        target = int(message.text.strip())
        # IMPORTANT:
        inject_fake_verification() writes straight into the
        # verifications table, which makes is_verified(target)=True even
        # if there's no row for this user in `users` yet. Without this
        # line, such a user would crash balance/withdraw with
        # "'NoneType' object is not subscriptable" the moment they tap a
        # dashboard button, because get_user() returns None.
        await
        DataEngine.create_user(target, "", "")
        await DataEngine.ban_user(target, 0)
        await
        DataEngine.inject_fake_verification(target)
        await state.clear()
        await message.answer(
            f"🚀 Full Unban done. User
<code>{target}</code> has direct menu
access.",

```

```

reply_markup=generate_admin_dashboard()
    )
    except ValueError:
        await message.answer("❌ Invalid
ID.")

#
_____
_____
_____

# FASTAPI – LIFESPAN
#
_____
_____
_____

@asynccontextmanager
async def application_lifespan(app:
FastAPI):
    global _polling_task, BOT_USERNAME
    await DataEngine.init_database()
    try:
        me = await bot.get_me()
        BOT_USERNAME = me.username or ""
    except Exception:
        logger.warning("[STARTUP] Could not
fetch bot username yet.")

    async def _run_polling():
        try:
            logger.info("[POLLING] Starting
Telegram long-polling...")
            await dp.start_polling(bot,
skip_updates=True)

```



```

        except Exception:
            logger.exception("[POLLING]
Polling task crashed and stopped! No
updates will be received until restart.")

        _polling_task =
        asyncio.create_task(_run_polling())
        yield

    api_platform =
    FastAPI(lifespan=application_lifespan)
    dp.include_router(core_router)

    _cors_origins = [ALLOWED_ORIGIN] if
    ALLOWED_ORIGIN else ["*"]
    api_platform.add_middleware(
        CORSMiddleware,
        allow_origins=_cors_origins,
        allow_credentials=bool(ALLOWED_ORIGIN),
        allow_methods=["POST", "GET"],
        allow_headers=["Content-Type"],
    )

#
_____
_____

# ROOT / HEALTH CHECK
#
# Railway (and anyone opening the
deployment URL directly in a browser) sends
# a plain GET request to "/". Without a
route registered for that path,
# FastAPI correctly returns 404

```

```
{"detail": "Not Found"} – this is NOT a  
crash,  
# it just looks alarming in a browser. This  
route makes GET "/" and GET  
# "/health" return a normal 200 OK so the  
deployment looks "alive", and it  
# also gives Railway's health checker  
something to succeed against.
```

```
#
```

```
@api_platform.get("/")  
async def root_health_check():  
    return {"status": "ok", "service":  
"referral-bot", "message": "Backend is  
running."}
```

```
@api_platform.get("/health")  
async def health_check():  
    return {"status": "ok"}
```

```
#
```

```
# MINI APP REST API
```

```
#
```

```
# Everything below is what the two frontend  
pages (index.html = verify  
# screen, app.html = dashboard/admin)  
actually call over HTTP. None of this  
# existed before – the FastAPI app had
```

```

middleware + lifespan but zero
# routes, so every Mini App request was a
silent 404.
#

```

```

class BaseAuthBody(BaseModel):
    initData: str = ""
    class Config:
        extra = "allow"

async def _authed_uid(body: BaseAuthBody) -
> int:
    tg_user =
parse_telegram_webapp_handshake(body.initDa
ta)
    if not tg_user or "id" not in tg_user:
        raise
    HTTPException(status_code=401,
detail="invalid_session")
    return int(tg_user["id"])

async def _authed_admin_uid(body:
BaseAuthBody) -> int:
    uid = await _authed_uid(body)
    if not evaluate_admin_access(uid):
        raise
    HTTPException(status_code=403,
detail="not_admin")
    return uid

```

```

async def _ensure_user_ok(uid: int,
tg_user: dict | None = None):
    acc = await DataEngine.get_user(uid)
    if not acc:
        uname = (tg_user or
{}).get("username", "")
        fname = f"{{(tg_user or
{}).get('first_name','')}} {{(tg_user or
{}).get('last_name','')}}".strip()
        await DataEngine.create_user(uid,
uname, fname)
        return
    if acc["is_banned"]:
        raise
HTTPException(status_code=403,
detail="banned")

async def _get_bot_username() -> str:
    global BOT_USERNAME
    if not BOT_USERNAME:
        me = await bot.get_me()
        BOT_USERNAME = me.username or ""
    return BOT_USERNAME

# — VERIFY (called from index.html)


---




---


class VerifyBody(BaseAuthBody):
    uid: int
    refId: int = 0
    msgId: int = 0
    ua: str = ""
    fingerprint: str = ""
    canvasHash: str = ""

```

```

webglHash: str = ""
screenSig: str = ""
tgPlatform: str = ""
tgVersion: str = ""
tgAppVersion: str = ""

@api_platform.post("/api/verify")
async def api_verify(body: VerifyBody,
request: Request):
    tg_user =
parse_telegram_webapp_handshake(body.initDa
ta)
    if not tg_user or int(tg_user.get("id",
0)) != body.uid:
        return
JSONResponse(status_code=400, content=
{"status": "blocked", "reason":
"no_fingerprint"})

    uid = body.uid
    ip = extract_real_ip(request)

    if not await
verify_limiter.is_allowed(f"verify:{uid}"):
        return
JSONResponse(status_code=429, content=
{"status": "blocked", "reason":
"ip_cooldown"})

    if await DataEngine.is_verified(uid):
        return {"status":
"already_verified"}

    acc = await DataEngine.get_user(uid)

```

```

        if acc and acc["is_banned"]:
            return {"status": "blocked",
                    "reason": "banned_ip"}

        if await DataEngine.is_ip_banned(ip):
            return {"status": "blocked",
                    "reason": "banned_ip"}

        on_cooldown, _remaining = await
        ip_cooldown.is_on_cooldown(ip)
        if on_cooldown:
            return {"status": "blocked",
                    "reason": "ip_cooldown"}

        if await
        execute_network_vpn_lookup(ip):
            return {"status": "blocked",
                    "reason": "vpn"}

        ref = body.refId if (body.refId and
        body.refId != uid) else 0
        uname = tg_user.get("username", "") or
        ""

        fname = f"
        {tg_user.get('first_name','')}
        {tg_user.get('last_name','')}".strip()

        should_ban, reason = await
        evaluate_clone_risk(
            new_user_id=uid, referrer_id=ref,
            client_ip=ip,
            fingerprint=body.fingerprint,
            tg_platform=body.tgPlatform,
            tg_version=body.tgVersion,

```

```

tg_app_version=body.tgAppVersion,
    canvas_hash=body.canvasHash,
webgl_hash=body.webglHash,
    screen_sig=body.screenSig,
)
    if should_ban:
        await DataEngine.create_user(uid,
uname, fname)
        await DataEngine.ban_user(uid, 1)
        await
DataEngine.log_fraud_attempt(uid, reason,
ip)
        if reason == "ip_farm":
            await DataEngine.ban_ip(ip,
reason)
            return {"status": "blocked",
"reason": reason}

        await DataEngine.create_user(uid,
uname, fname, referred_by=(ref or None))
        await DataEngine.save_verification(
            uid, ip, body.ua, body.fingerprint,
referrer_ip="",
            tg_platform=body.tgPlatform,
tg_version=body.tgVersion,
tg_app_version=body.tgAppVersion,
            canvas_hash=body.canvasHash,
webgl_hash=body.webglHash,
screen_sig=body.screenSig,
        )
        await ip_cooldown.mark_verified(ip)

    if ref:
        rate = float(await

```

```

DataEngine.get_setting("reward_per_referral", "10"))
        await DataEngine.add_balance(ref,
rate)
        try:
            await bot.send_message(ref,
f"🎉 New referral joined! +{rate:.2f} Birr credited.")
        except Exception:
            pass

        if body.msgId:
            try:
                await
bot.delete_message(chat_id=uid,
message_id=body.msgId)
            except Exception:
                pass
        try:
            await bot.send_message(uid, "✅
Identity clear! Welcome.",
reply_markup=generate_dashboard_matrix(uid)
)
        except Exception:
            pass

        return {"status": "verified"}

# — ME / DASHBOARD (app.html)

@api_platform.post("/api/me")
async def api_me(body: BaseAuthBody):
    uid = await _authed_uid(body)

```



```

        await _ensure_user_ok(uid,
parse_telegram_webapp_handshake(body.initDa
ta))
        await DataEngine.touch_last_seen(uid)
        acc = await DataEngine.get_user(uid)
        direct, tier2 = await
DataEngine.get_referral_metrics(uid)
        rate = float(await
DataEngine.get_setting("reward_per_referral
", "10"))
        min_w = float(await
DataEngine.get_setting("min_withdrawal",
"50"))
        uname = await _get_bot_username()
        return {
            "balance": float(acc["balance"]),
            "is_admin":
evaluate_admin_access(uid),
            "referrals": direct,
            "tier2_referrals": tier2,
            "reward_per_referral": rate,
            "total_earned_refs": direct * rate,
            "min_withdrawal": min_w,
            "referral_link":
f"https://t.me/{uname}?start={uid}",
        }

```

— GATE CHANNELS (Mini App entry)

'force' channels (bot_added=0): the bot
IS an admin there, so membership is
really checked via getChatMember. These
block the Mini App dashboard until

```

# joined, and can be re-checked ("I've
Joined") – if the user left, they're
# asked to join again.
# 'fake' channels (bot_added=1): the bot
can't verify membership, so these
# are never used to block anything. They're
surfaced once in the Tasks tab
# as a simple "view/visit" item
(fake_join_seen), never re-prompted.
@api_platform.post("/api/gate/status")
async def api_gate_status(body:
BaseAuthBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    all_channels = await
DataEngine.get_force_channels()
    force_unjoined = []
    fake_channels = []
    for ch in all_channels:
        if ch["bot_added"] == 1:
            fake_channels.append({
                "channel_id":
ch["channel_id"], "channel_name":
ch["channel_name"],
                "invite_link":
ch["invite_link"],
            })
            continue
        try:
            m = await
bot.get_chat_member(chat_id=ch["channel_id"
], user_id=uid)
            if m.status in
(ChatMemberStatus.LEFT,

```

```

ChatMemberStatus.KICKED,
ChatMemberStatus.RESTRICTED):
    force_unjoined.append({
        "channel_id":
ch["channel_id"], "channel_name":
ch["channel_name"],
        "invite_link":
ch["invite_link"],
    })
    except Exception as e:
        # Unlike the bot-chat gate
        (which skips on error so a misconfigured
        # channel never locks out every
        user everywhere), the Mini App
        # gate is meant to be a hard
        wall: if a Force channel can't be
        # verified – usually because
        the bot isn't an admin there yet –
        # we block rather than silently
        let people through.
        # ⚠️ Make sure the bot is an
        admin in every Force channel, or
        # this will lock EVERYONE out
        of the dashboard.
        logger.warning(
            f"[GATE] Could not verify
membership for channel={ch['channel_id']} "
            f"user={uid}: {e} –
blocking (Mini App gate fails closed)."
        )
        force_unjoined.append({
            "channel_id":
ch["channel_id"], "channel_name":
ch["channel_name"],

```

```

        "invite_link":
ch["invite_link"],
        })
        fake_seen = await
DataEngine.has_seen_fake_join(uid)
        return {
            "force_unjoined": force_unjoined,
            "fake_channels": fake_channels,
            "fake_seen": fake_seen,
        }

@api_platform.post("/api/gate/fake_seen")
async def api_gate_fake_seen(body:
BaseAuthBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    await
DataEngine.mark_fake_join_seen(uid)
    return {"ok": True}

# — TASKS


---




---


class TaskActionBody(BaseAuthBody):
    task_id: int

@api_platform.post("/api/tasks")
async def api_tasks(body: BaseAuthBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    tasks = await
DataEngine.get_tasks(active_only=True)
    progress = await
DataEngine.get_all_task_progress_for_user(u

```

```

id)
    now = datetime.utcnow()
    out = []
    for t in tasks:
        p = progress.get(t["id"])
        if p is None:
            status, wait_left =
"available", 0
        elif p["status"] == "completed":
            status, wait_left =
"completed", 0
        else:
            try:
                joined_at =
datetime.fromisoformat(p["joined_at"])
                elapsed = (now -
joined_at).total_seconds()
            except Exception:
                elapsed =
TASK_JOIN_WAIT_SECONDS
            remaining = max(0,
int(TASK_JOIN_WAIT_SECONDS - elapsed))
            status = "claimable" if
remaining <= 0 else "waiting"
            wait_left = remaining
            out.append({
                "id": t["id"], "title":
t["title"], "reward": float(t["reward"]),
                "task_type": t["task_type"],
                "invite_link": t["invite_link"],
                "status": status, "wait_left":
wait_left,
            })
    return {"tasks": out}

```

```

@api_platform.post("/api/tasks/join")
async def api_tasks_join(body:
TaskActionBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    task = await
DataEngine.get_task(body.task_id)
    if not task or not task["is_active"]:
        raise
HTTPException(status_code=404,
detail="not_found")
    await DataEngine.mark_task_joined(uid,
body.task_id)
    return {"wait_seconds":
TASK_JOIN_WAIT_SECONDS}

@api_platform.post("/api/tasks/check")
async def api_tasks_check(body:
TaskActionBody):
    """
    Lightweight verification step –
    confirms the wait timer is done and
    (for 'force' tasks) that the user has
    actually joined the channel via
    a real getChatMember lookup. Does NOT
    pay out the reward; that only
    happens in /api/tasks/claim. This lets
    the Mini App show a genuine
    Join → Check → Claim flow instead of
    paying blindly after a timer.
    """
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)

```

```

        task = await
DataEngine.get_task(body.task_id)
        if not task:
            raise
HTTPException(status_code=404,
detail="not_found")
        progress = await
DataEngine.get_task_progress(uid,
body.task_id)
        if not progress or progress["status"]
== "completed":
            raise
HTTPException(status_code=400,
detail="not_joined")
        try:
            joined_at =
datetime.fromisoformat(progress["joined_at"
])
            elapsed = (datetime.utcnow() -
joined_at).total_seconds()
        except Exception:
            elapsed = TASK_JOIN_WAIT_SECONDS
            remaining = int(TASK_JOIN_WAIT_SECONDS
- elapsed)
            if remaining > 0:
                raise
HTTPException(status_code=400,
detail=f"wait:{remaining}")
            if task["task_type"] == "force" and
task["channel_id"]:
                try:
                    m = await
bot.get_chat_member(chat_id=task["channel_i
d"], user_id=uid)

```

```

        if m.status in
(ChatMemberStatus.LEFT,
ChatMemberStatus.KICKED,
ChatMemberStatus.RESTRICTED):
            raise
HTTPException(status_code=400,
detail="not_member")
        except HTTPException:
            raise
        except Exception:
            logger.warning(f"[TASK] Could
not verify membership for task={task['id']}
user={uid} – allowing through.")
            return {"status": "verified"}

@api_platform.post("/api/tasks/claim")
async def api_tasks_claim(body:
TaskActionBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    task = await
DataEngine.get_task(body.task_id)
    if not task:
        raise
HTTPException(status_code=404,
detail="not_found")
    progress = await
DataEngine.get_task_progress(uid,
body.task_id)
    if not progress or progress["status"]
== "completed":
        raise
HTTPException(status_code=400,
detail="not_joined")

```



```

        try:
            joined_at =
datetime.fromisoformat(progress["joined_at"
])
            elapsed = (datetime.utcnow() -
joined_at).total_seconds()
            except Exception:
                elapsed = TASK_JOIN_WAIT_SECONDS
                remaining = int(TASK_JOIN_WAIT_SECONDS
- elapsed)
                if remaining > 0:
                    raise
HTTPException(status_code=400,
detail=f"wait:{remaining}")
                if task["task_type"] == "force" and
task["channel_id"]:
                    try:
                        m = await
bot.get_chat_member(chat_id=task["channel_i
d"], user_id=uid)
                        if m.status in
(ChatMemberStatus.LEFT,
ChatMemberStatus.KICKED,
ChatMemberStatus.RESTRICTED):
                            raise
HTTPException(status_code=400,
detail="not_member")
                    except HTTPException:
                        raise
                    except Exception:
                        logger.warning(f"[TASK] Could
not verify membership for task={task['id']}
user={uid} – allowing through.")
                        await

```

```
DataEngine.mark_task_completed(uid,
body.task_id)
    reward = float(task["reward"])
    await DataEngine.add_balance(uid,
reward)
    acc = await DataEngine.get_user(uid)
    return {"status": "completed",
"reward": reward, "balance":
float(acc["balance"])}
```

```
# — WITHDRAW
```

```
class WithdrawBody(BaseAuthBody):
    amount: float
    phone: str
    full_name: str

@api_platform.post("/api/withdraw")
async def api_withdraw(body: WithdrawBody):
    uid = await _authed_uid(body)
    await _ensure_user_ok(uid)
    min_w = float(await
DataEngine.get_setting("min_withdrawal",
"50"))
    if body.amount < min_w:
        raise
    HTTPException(status_code=400,
detail="below_minimum")
    if len(body.phone.strip()) < 9:
        raise
    HTTPException(status_code=400,
detail="invalid_phone")
    if len(body.full_name.strip()) < 3:
```

```

        raise
    HTTPException(status_code=400,
detail="invalid_name")
    result = await
dispatch_withdrawal_core(uid,
round(body.amount, 2),
body.full_name.strip(), body.phone.strip())
    if result["status"] == "error":
        raise
    HTTPException(status_code=400,
detail="insufficient_funds")
    return {"status": "submitted",
"ticket_id": result["ticket_id"]}

@api_platform.post("/api/withdraw/history")
async def api_withdraw_history(body:
BaseAuthBody):
    uid = await _authed_uid(body)
    rows = await
DataEngine.get_user_withdrawals(uid)
    return {"withdrawals": [
        {"amount": float(r["amount"]),
"status": r["status"], "created_at":
r["created_at"]}
        for r in rows
    ]}

# — ADMIN: OVERVIEW


---




---



@api_platform.post("/api/admin/stats/overvi
ew")
async def api_admin_stats_overview(body:
BaseAuthBody):

```

```

        await _authed_admin_uid(body)
        total, online = await
DataEngine.get_user_activity_stats(online_w
indow_minutes=5)
        return {"total_users": total,
"online_users": online}

# — ADMIN: TASKS


---




---


class IdBody(BaseAuthBody):
    id: int

class AdminTaskCreateBody(BaseAuthBody):
    title: str
    task_type: str
    channel_id: str = ""
    invite_link: str = ""
    reward: float = 0

class AdminTaskUpdateBody(BaseAuthBody):
    id: int
    is_active: Optional[int] = None

class ReorderBody(BaseAuthBody):
    order: List[int]

@api_platform.post("/api/admin/tasks/list")
async def api_admin_tasks_list(body:
BaseAuthBody):
    await _authed_admin_uid(body)
    tasks = await
DataEngine.get_tasks(active_only=False)
    return {"tasks": [dict(t) for t in

```

```

tasks]]

@api_platform.post("/api/admin/tasks/create
")
async def api_admin_tasks_create(body:
AdminTaskCreateBody):
    await _authed_admin_uid(body)
    tid = await
DataEngine.create_task(body.title,
body.channel_id, body.invite_link,
body.task_type, body.reward)
    return {"id": tid}

@api_platform.post("/api/admin/tasks/delete
")
async def api_admin_tasks_delete(body:
IdBody):
    await _authed_admin_uid(body)
    await DataEngine.delete_task(body.id)
    return {"ok": True}

@api_platform.post("/api/admin/tasks/update
")
async def api_admin_tasks_update(body:
AdminTaskUpdateBody):
    await _authed_admin_uid(body)
    fields = {}
    if body.is_active is not None:
        fields["is_active"] =
body.is_active
    await DataEngine.update_task(body.id,
**fields)
    return {"ok": True}

```

```

@api_platform.post("/api/admin/tasks/reorder")
async def api_admin_tasks_reorder(body:
ReorderBody):
    await _authed_admin_uid(body)
    await
DataEngine.reorder_tasks(body.order)
    return {"ok": True}

# — ADMIN: GATE CHANNELS


---




---


class AdminChannelCreateBody(BaseAuthBody):
    bot_added: int
    channel_id: str = ""
    channel_name: str
    invite_link: str

@api_platform.post("/api/admin/channels/list")
async def api_admin_channels_list(body:
BaseAuthBody):
    await _authed_admin_uid(body)
    channels = await
DataEngine.get_force_channels()
    return {"channels": [dict(c) for c in
channels]}

@api_platform.post("/api/admin/channels/create")
async def api_admin_channels_create(body:
AdminChannelCreateBody):
    await _authed_admin_uid(body)
    cid = body.channel_id.strip()

```

```

        if body.bot_added == 1 and not cid:
            cid = "fake_" +
hashlib.md5(body.invite_link.encode()).hexdigest()[:8]
            await DataEngine.add_force_channel(cid,
body.channel_name, body.invite_link,
bot_added=body.bot_added)
            return {"ok": True}

@api_platform.post("/api/admin/channels/delete")
async def api_admin_channels_delete(body:
IdBody):
    await _authed_admin_uid(body)
    await
DataEngine.remove_force_channel(body.id)
    return {"ok": True}

# — ADMIN: WITHDRAWALS

```

```

class RejectBody(BaseAuthBody):
    id: int
    reason: str = "Violated bot usage
policies."

@api_platform.post("/api/admin/withdrawals/
list")
async def api_admin_withdrawals_list(body:
BaseAuthBody):
    await _authed_admin_uid(body)
    rows = await
DataEngine.get_pending_withdrawals()
    return {"withdrawals": [dict(r) for r

```

```

in rows]]

@api_platform.post("/api/admin/withdrawals/
approve")
async def
api_admin_withdrawals_approve(body:
IdBody):
    await _authed_admin_uid(body)
    result = await
approve_withdrawal_core(body.id)
    if not result["ok"]:
        raise
    HTTPException(status_code=400,
    detail=result["reason"])
    return {"ok": True}

@api_platform.post("/api/admin/withdrawals/
reject")
async def
api_admin_withdrawals_reject(body:
RejectBody):
    await _authed_admin_uid(body)
    ticket = await
DataEngine.get_withdrawal(body.id)
    if not ticket or ticket["status"] !=
    "pending":
        raise
    HTTPException(status_code=400,
    detail="already_processed")
    await
execute_withdrawal_rejection(None, body.id,
ticket, body.reason)
    return {"ok": True}

```



```
# — ADMIN: USERS
```

```
class UserIdBody(BaseAuthBody):  
    user_id: int
```

```
class BanBody(BaseAuthBody):  
    user_id: int  
    banned: int
```

```
class BalanceAdjustBody(BaseAuthBody):  
    user_id: int  
    amount: float
```

```
@api_platform.post("/api/admin/users/search  
")
```

```
async def api_admin_users_search(body:  
    UserIdBody):  
    await _authed_admin_uid(body)  
    user = await  
DataEngine.get_user(body.user_id)  
    if not user:  
        raise  
    HTTPException(status_code=404,  
    detail="not_found")  
    direct, _tier2 = await  
DataEngine.get_referral_metrics(body.user_i  
d)  
    return {  
        "full_name": user["full_name"] or  
    "",  
        "username": user["username"] or "",  
        "balance": float(user["balance"]),  
        "referrals": direct,
```

```

        "is_banned": user["is_banned"],
    }

@api_platform.post("/api/admin/users/ban")
async def api_admin_users_ban(body:
BanBody):
    await _authed_admin_uid(body)
    await DataEngine.ban_user(body.user_id,
body.banned)
    if body.banned:
        try:
            await
bot.send_message(body.user_id, "🚫 <b>Your
account has been banned from this bot.
</b>")
        except Exception:
            pass
    return {"ok": True}

@api_platform.post("/api/admin/users/balanc
e")
async def api_admin_users_balance(body:
BalanceAdjustBody):
    await _authed_admin_uid(body)
    await
DataEngine.add_balance(body.user_id,
body.amount)
    return {"ok": True}

# — ADMIN: SETTINGS


---




---


class SettingsUpdateBody(BaseAuthBody):
    reward_per_referral: float

```

```

        min_withdrawal: float

@api_platform.post("/api/admin/settings")
async def api_admin_settings(body:
BaseAuthBody):
    await _authed_admin_uid(body)
    reward = await
DataEngine.get_setting("reward_per_referral
", "10")
    min_w = await
DataEngine.get_setting("min_withdrawal",
"50")
    return {"reward_per_referral":
float(reward), "min_withdrawal":
float(min_w)}

@api_platform.post("/api/admin/settings/upd
ate")
async def api_admin_settings_update(body:
SettingsUpdateBody):
    await _authed_admin_uid(body)
    await
DataEngine.set_setting("reward_per_referral
", str(body.reward_per_referral))
    await
DataEngine.set_setting("min_withdrawal",
str(body.min_withdrawal))
    return {"ok": True}

# — ADMIN: BROADCAST

```

```

class BroadcastBody(BaseAuthBody):
    text: str

```

```
@api_platform.post("/api/admin/broadcast")
async def api_admin_broadcast(body:
BroadcastBody):
    await _authed_admin_uid(body)
    async with aiosqlite.connect(DB_PATH)
as db:
        cur = await db.execute("SELECT
user_id FROM users")
        nodes = await cur.fetchall()
        sent, failed = 0, 0
        for (target_uid,) in nodes:
            try:
                await
bot.send_message(target_uid, body.text)
                sent += 1
                await asyncio.sleep(0.05)
            except Exception:
                failed += 1
        return {"sent": sent, "failed": failed}

if __name__ == "__main__":
    port = int(os.environ.get("PORT",
8000))
    uvicorn.run(api_platform,
host="0.0.0.0", port=port)

#
_____
_____
_____
```

